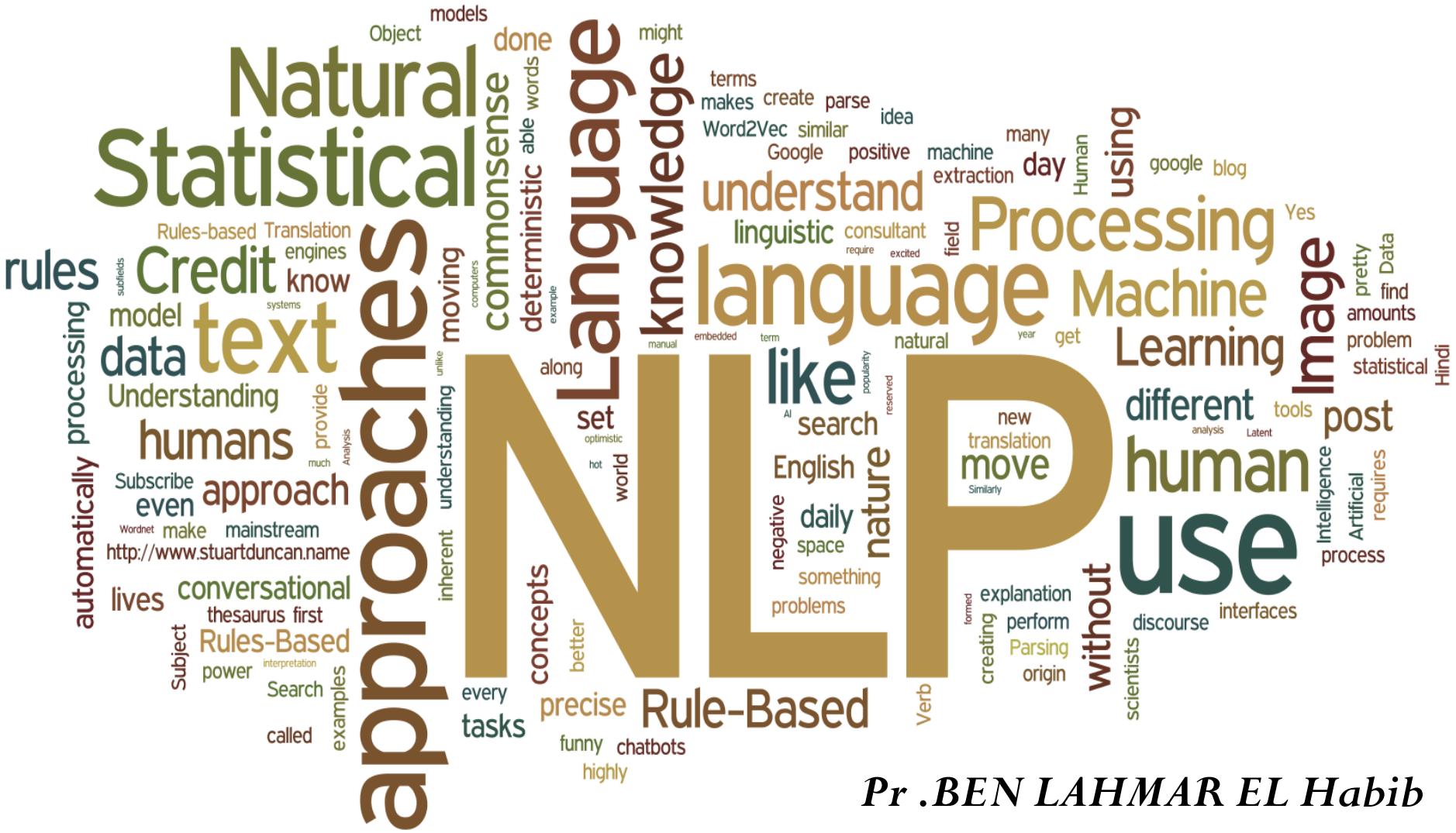
FSBM

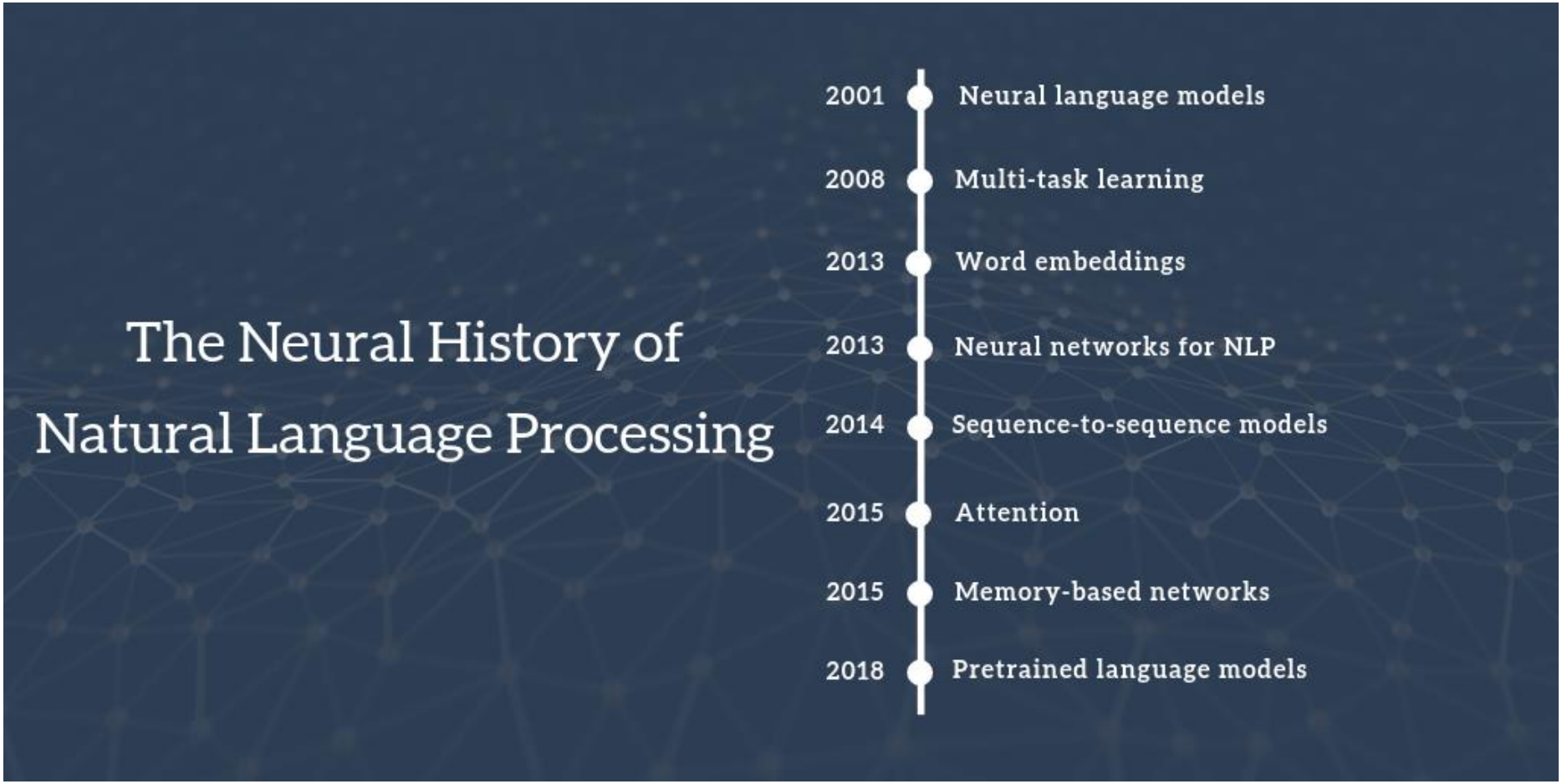
FACULTE DES SCIENCES BEN M'SICK

le NLP à l'ére de l'IA.



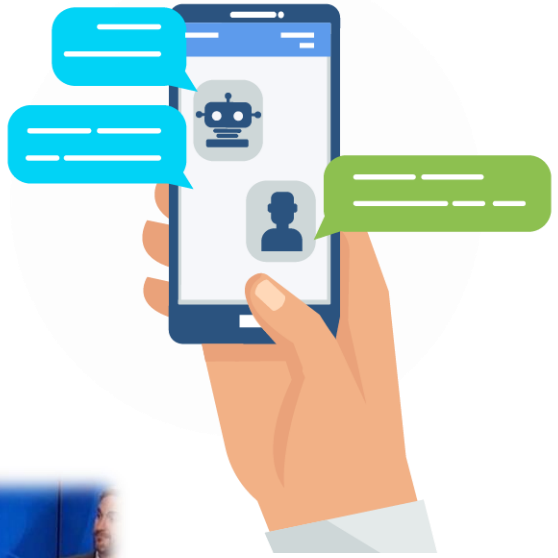
Pr .BEN LAHMAR EL Habib

Evolution


The Neural History of
Natural Language Processing
2001 Neural language models
2008 Multi-task learning
2013 Word embeddings
2013 Neural networks for NLP
2014 Sequence-to-sequence models
2015 Attention
2015 Memory-based networks
2018 Pretrained language models

Le traitement du Langage naturel









1954



L'expérience Georgetown – IBM
Traduction automatique
le 7 janvier 1954

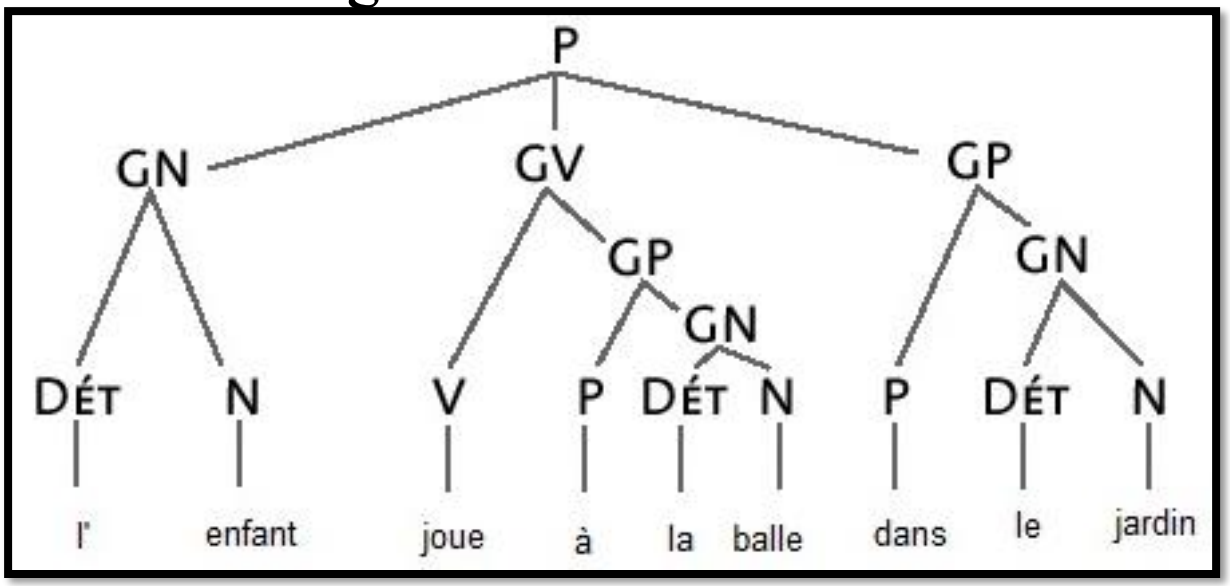


agent conversationnel
1964 -1966

Approche basée sur des règles

- **Analyse** de la phrase
- **Transfert** vers le langage destination
- **Génération** du texte traduit

Nécessite des **dictionnaires** et des **règles**

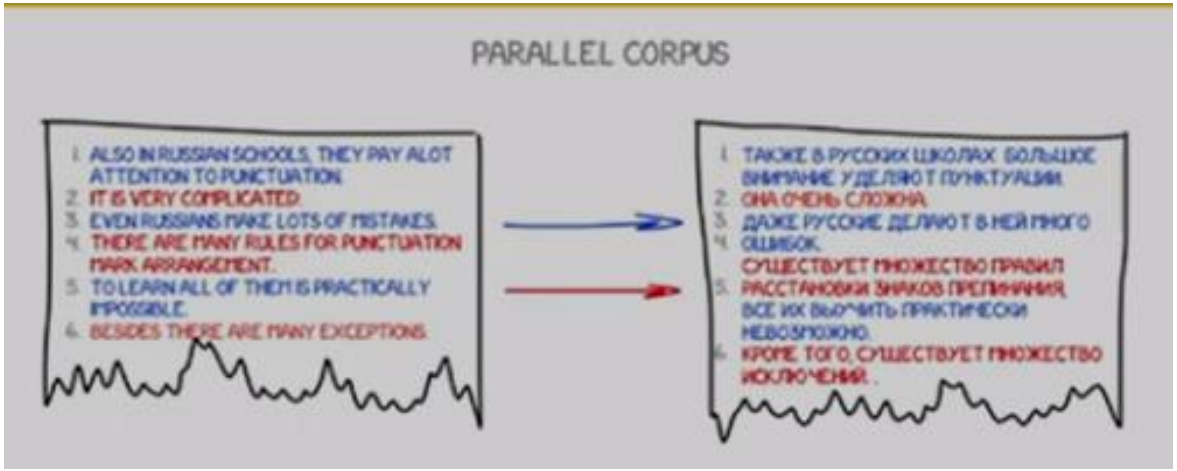


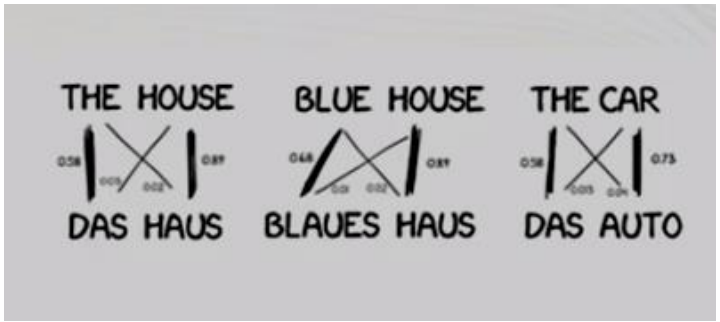
L'ambiguïté

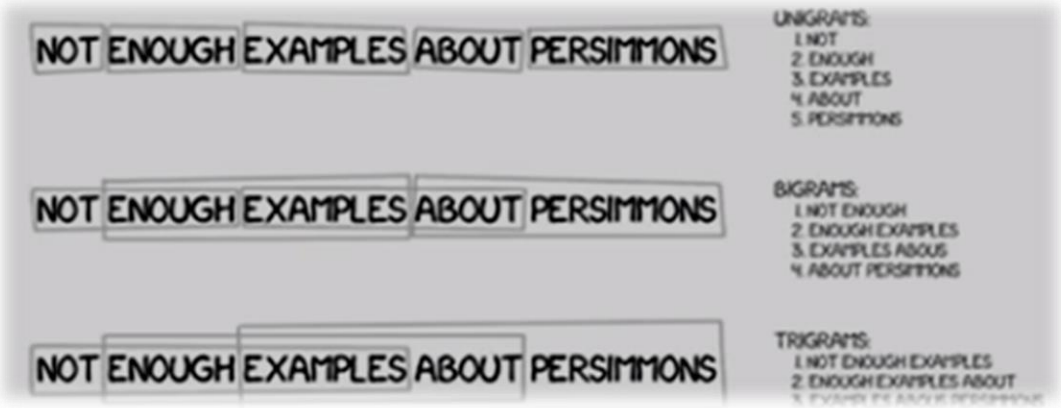
J'ai vu Driss avec des jumelles

Contexte

Approches statistiques



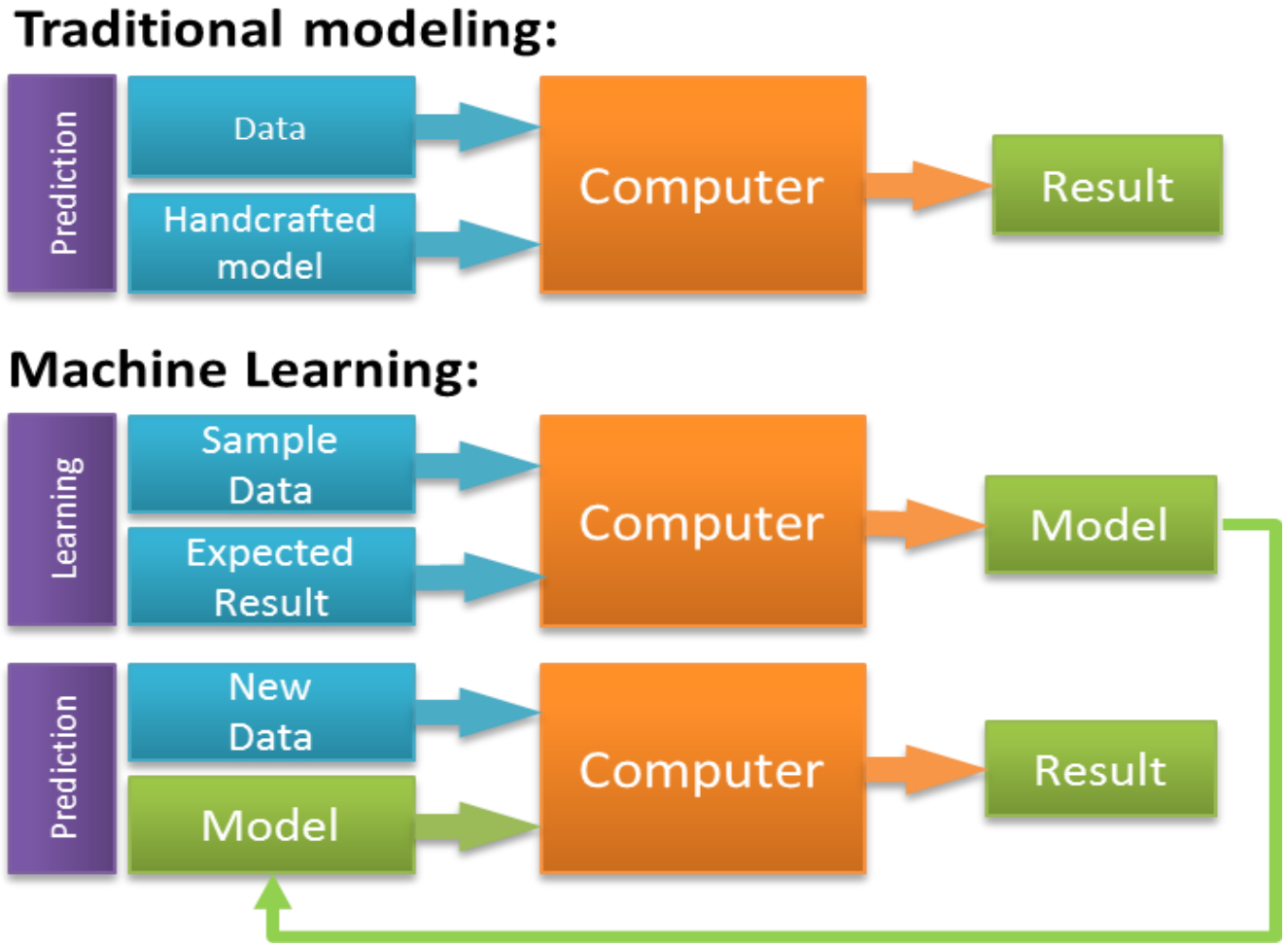




N-Gram

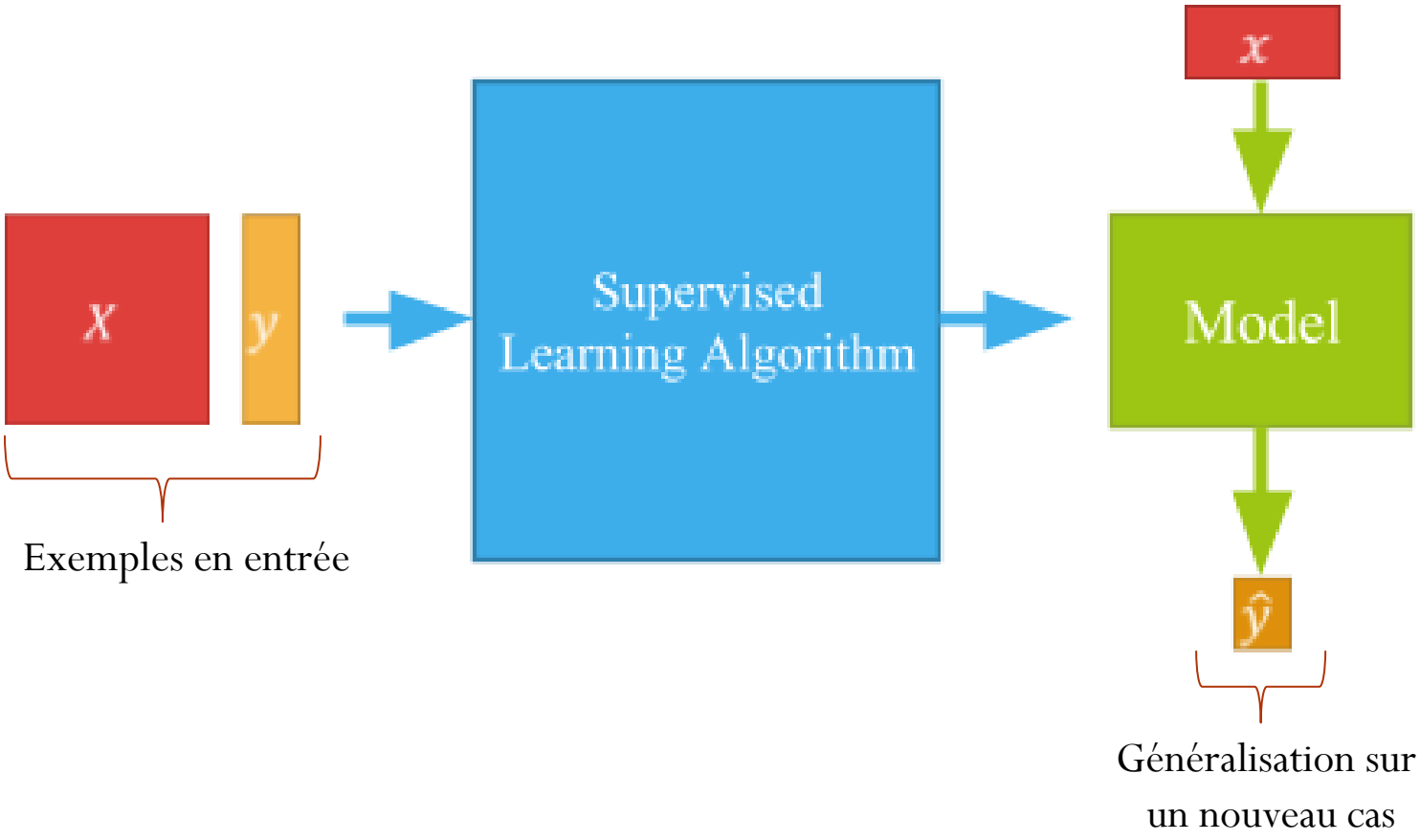
Machine Learning

- Apprendre à modéliser un problème à partir des données

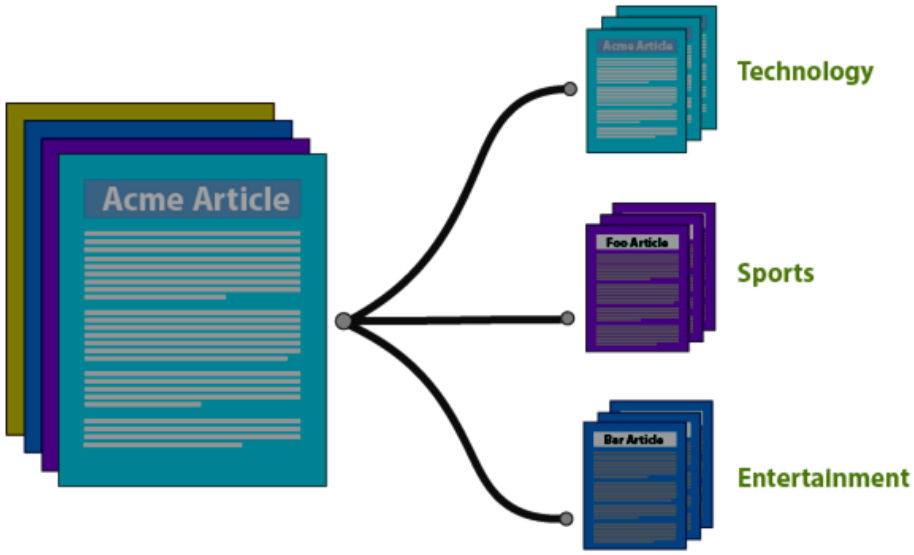


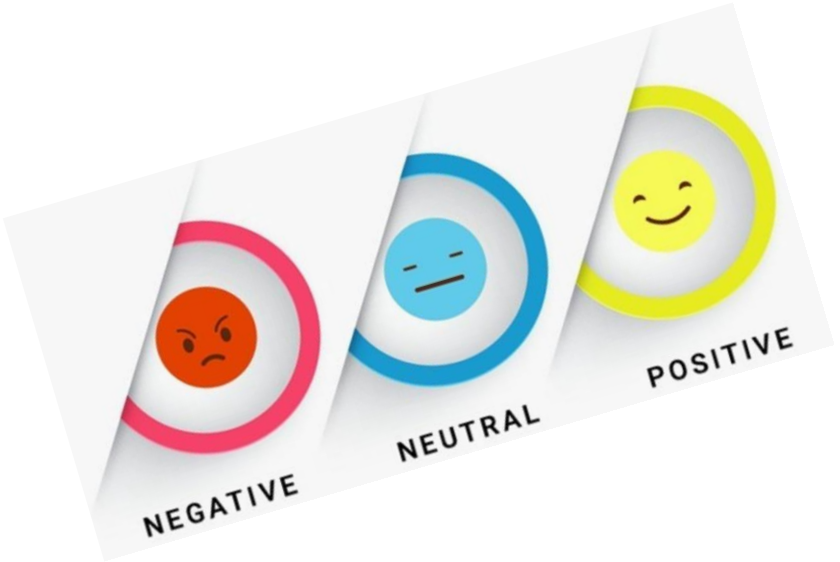
Apprentissage supervisé

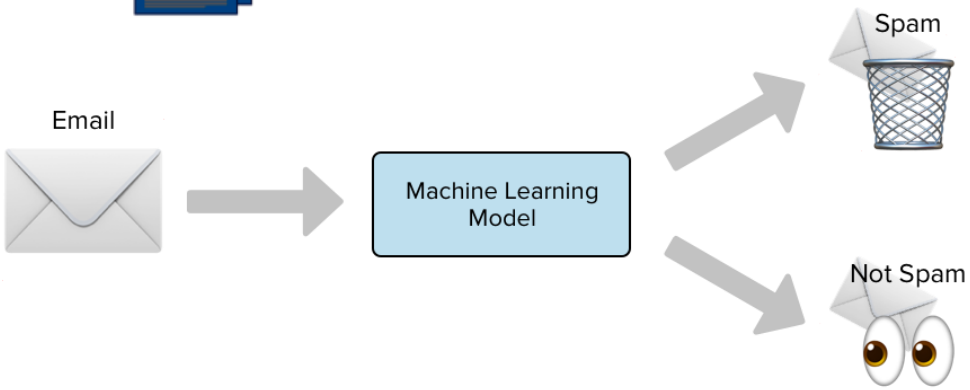
- Observer des exemples et produire des généralisation



Classification de texte





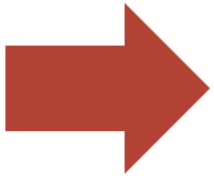


Comment représenter du texte pour que l'algorithme puisse l'analyser ?

Sac des mots … Bag of Words

- *Représenter les mots ou les phrases d'un texte par des vecteurs de nombres réels, décrits dans un modèle vectoriel (Vector Space Model).*

Vocabulary:
Man, woman, boy, girl, prince, princess, queen, king, monarch



	1	2	3	4	5	6	7	8	9
man	1	0	0	0	0	0	0	0	0
woman	0	1	0	0	0	0	0	0	0
boy	0	0	1	0	0	0	0	0	0
girl	0	0	0	1	0	0	0	0	0
prince	0	0	0	0	1	0	0	0	0
princess	0	0	0	0	0	1	0	0	0
queen	0	0	0	0	0	0	1	0	0
king	0	0	0	0	0	0	0	1	0
monarch	0	0	0	0	0	0	0	0	1

One-Hot Encoding

Exemple

- Phrase 1: Les mangues sont jaunes.
- Phrase 2: Les pommes sont rouges.

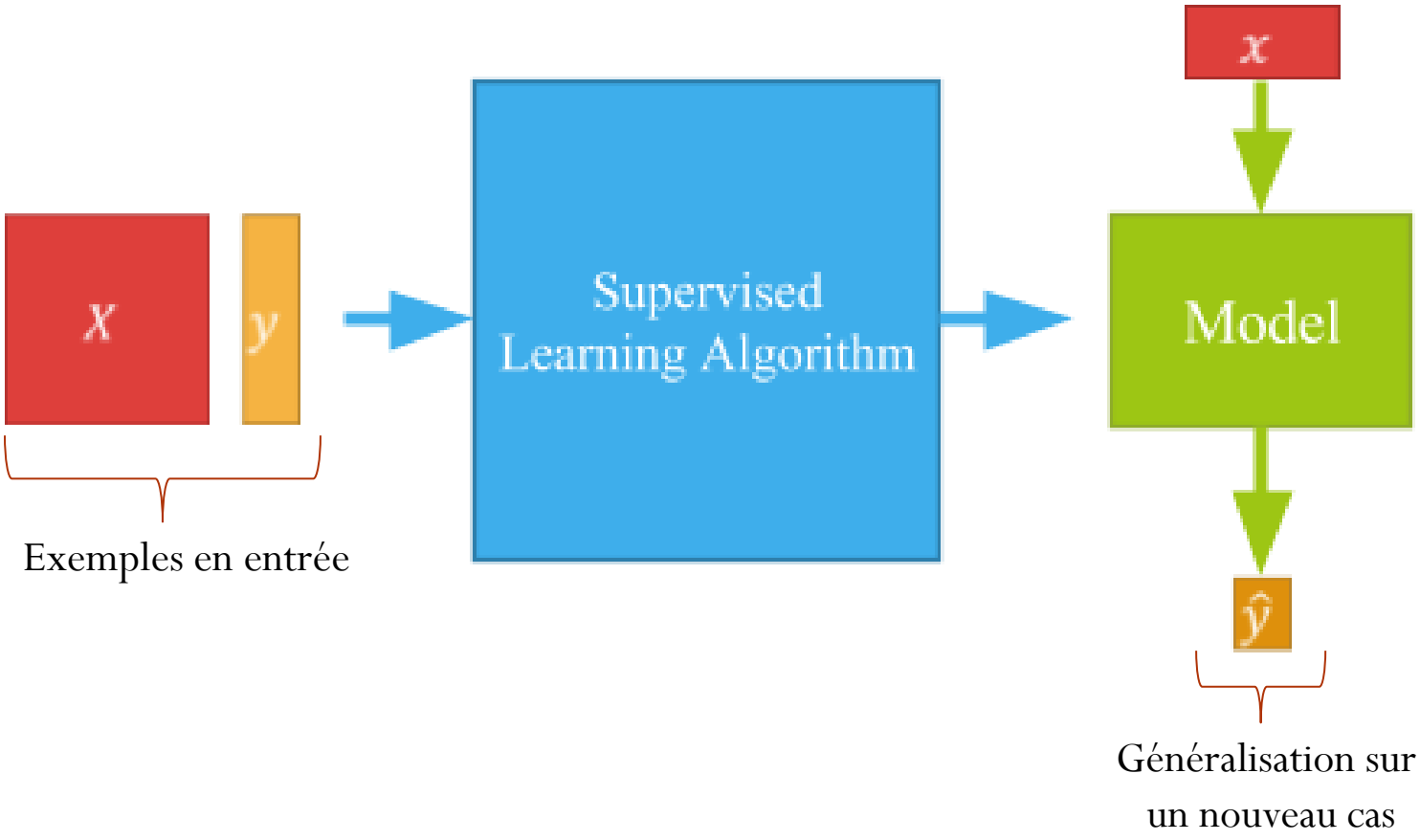


[1,0,0,0,0,0] **[0,1,0,0,0,0]** **[0,0,1,0,0,0]**

- Si les mots de notre corpus sont {Les, mangues, sont, jaune, pommes, rouge}.
- la phrase 1 sera représentée par **[1,1,1,1,0,0]**
- et la phrase 2 sera **[1,0,1,0,1,1]**.

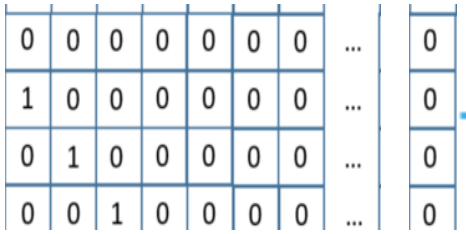
Apprentissage supervisé

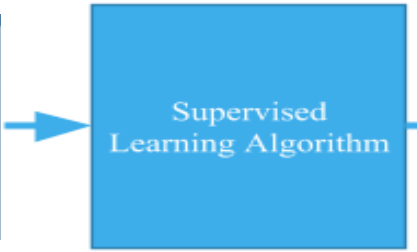
- Observer des exemples et produire des généralisation

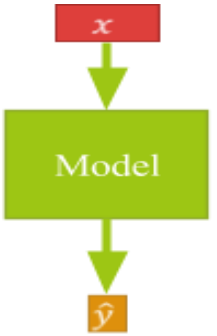


Classification

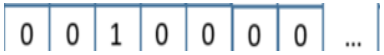
review	sentiment
One of the other reviewers has mentioned that ...	positive
A wonderful little production. The...	positive
I thought this was a wonderful way to spend ti...	positive
Basically there's a family where a little boy ...	negative
Petter Mattei's "Love in the Time of Money" is...	positive







Nouvelle phrase

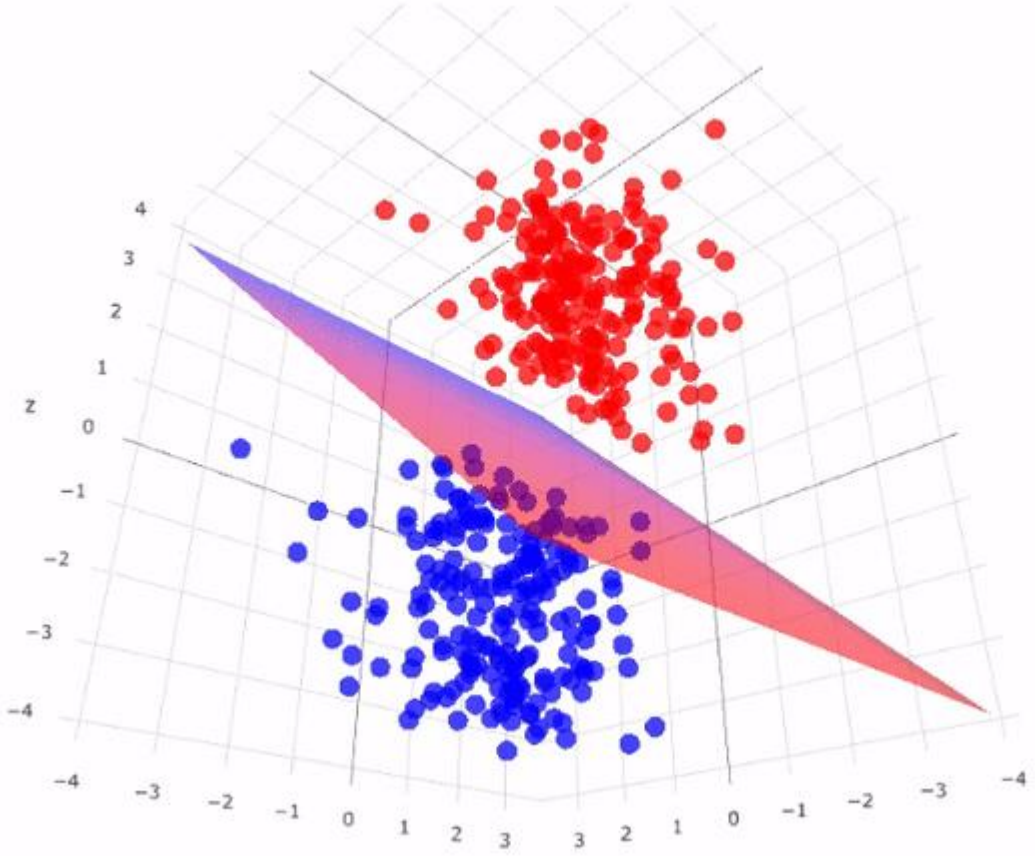


Prédiction



DataSet labels

Modèles de classification



SVM

Régression logistique

Pré-traitement

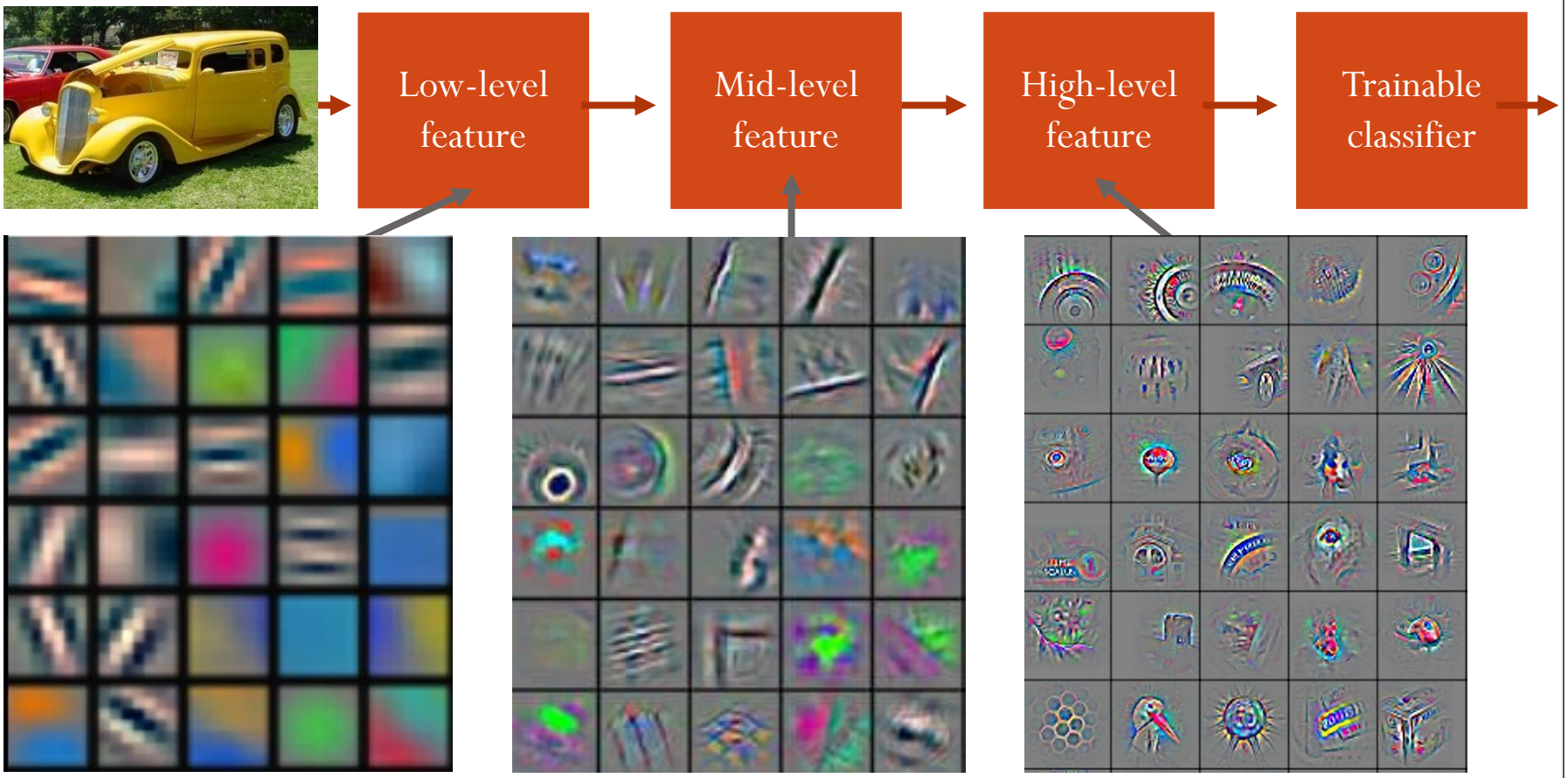
- Étiquetage
- Lemmatisation
- Fréquence
- Mots-vide
- …

NLP avec du Deep Learning



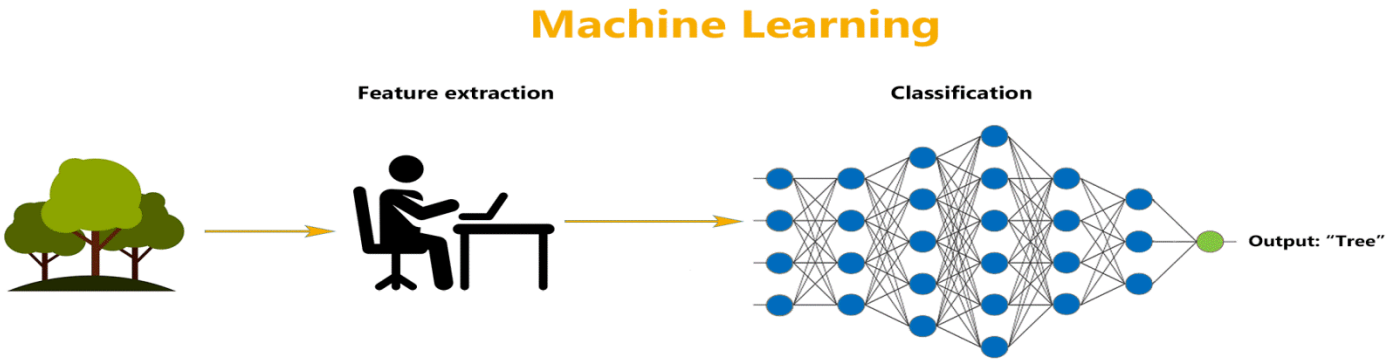
Deep learning = learning hierarchical representations

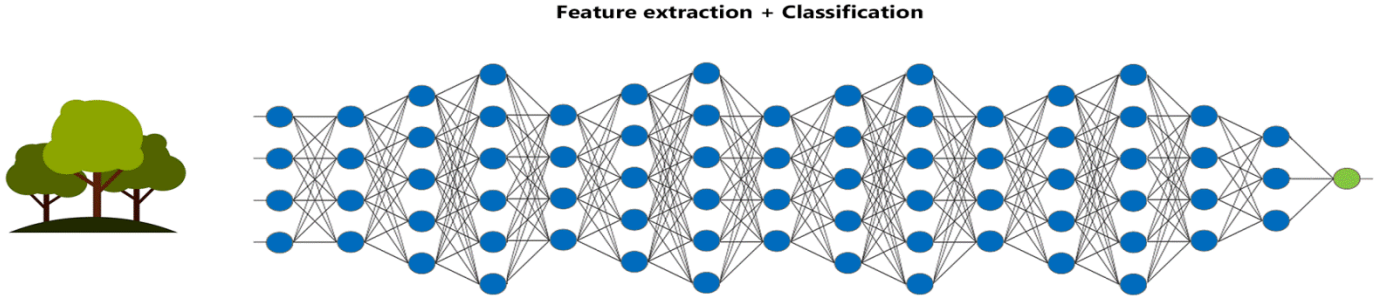
It's deep if it has more than one stage of non-linear feature transformation



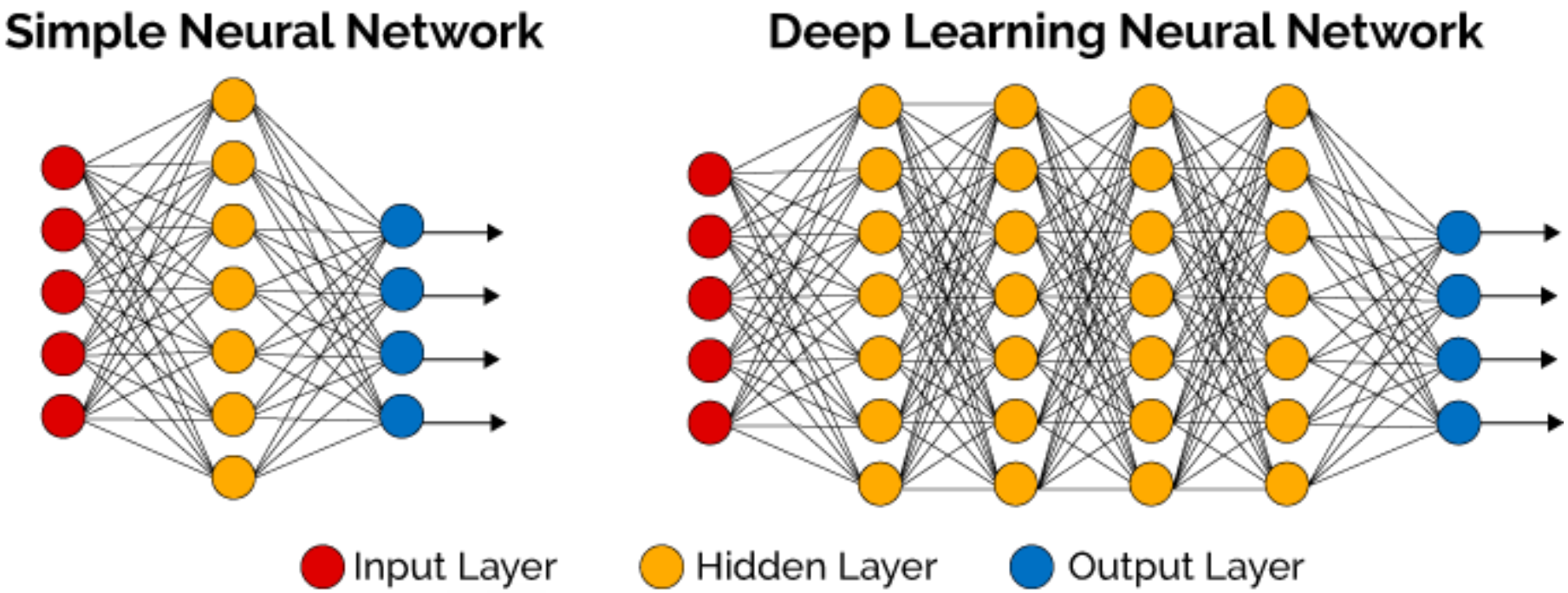
Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

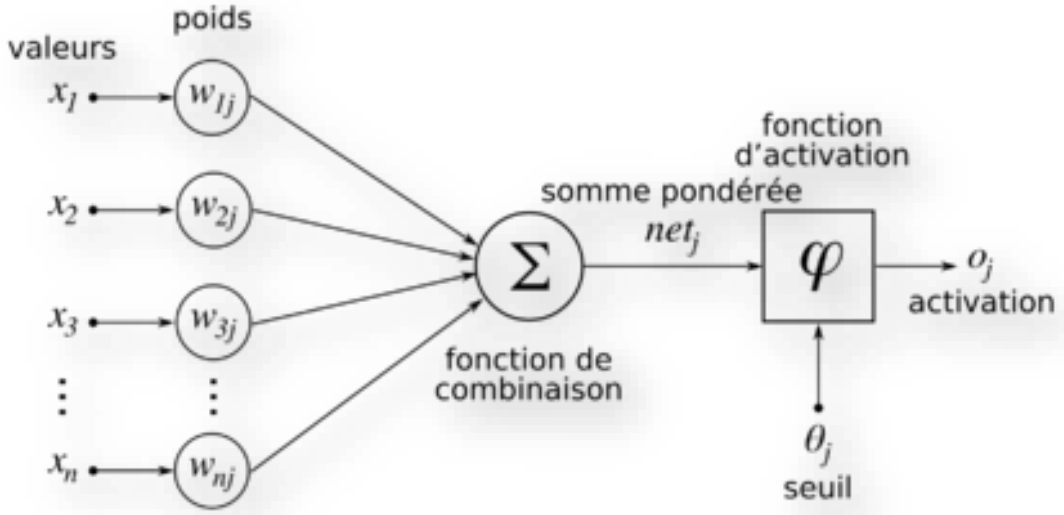
Deep Learning





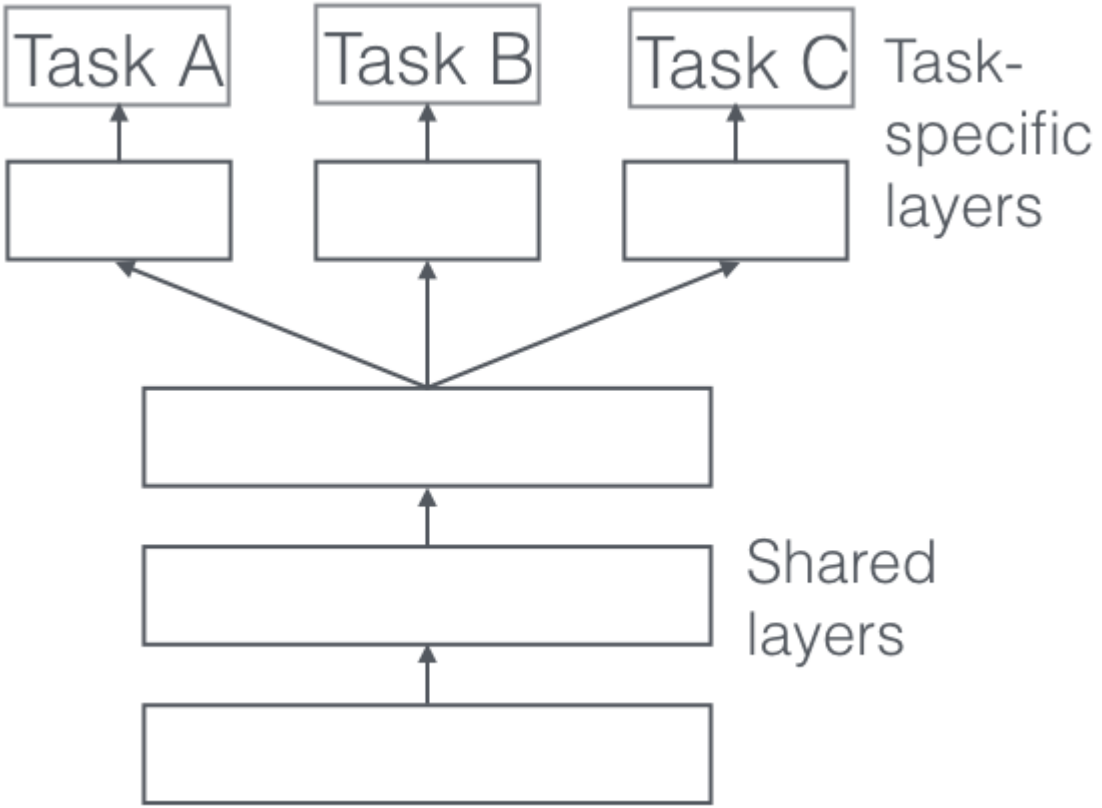
ANN

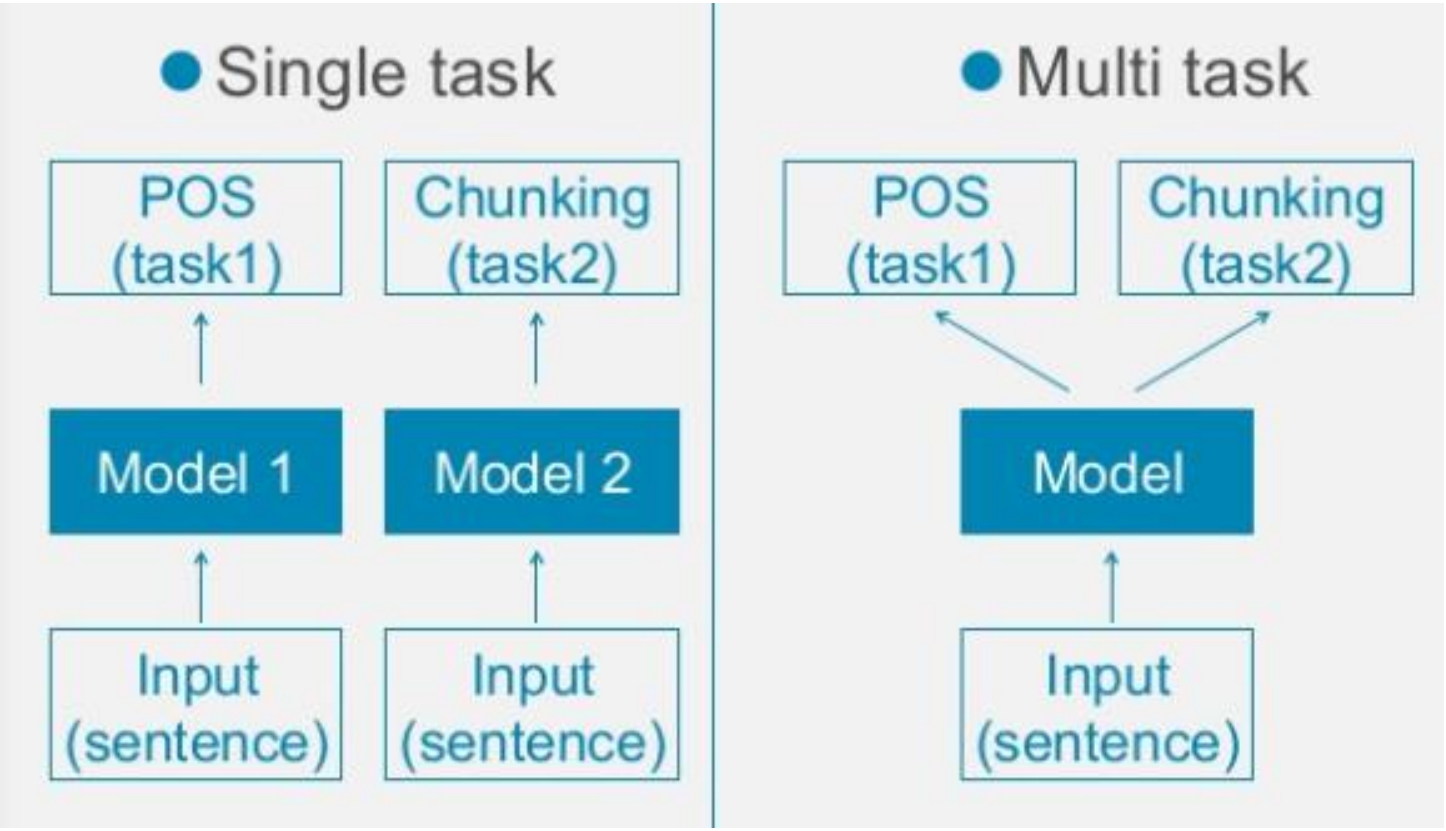

Simple Neural Network
Deep Learning Neural Network
Input Layer
Hidden Layer
Output Layer


poids
valeurs
x_1
x_2
x_3
x_n
w_{1j}
w_{2j}
w_{3j}
w_{nj}
Σ
somme pondérée
net_j
fonction de combinaison
fonction d'activation
φ
o_j
activation
θ_j
seuil

2008 - Multi-task learning

- L'apprentissage multitâche est une méthode générale pour partager des paramètres entre des modèles formés sur plusieurs tâches.




Single task
POS (task1)
Chunking (task2)
Model 1
Model 2
Input (sentence)
Input (sentence)
Multi task
POS (task1)
Chunking (task2)
Model
Input (sentence)

2013 - Word embeddings

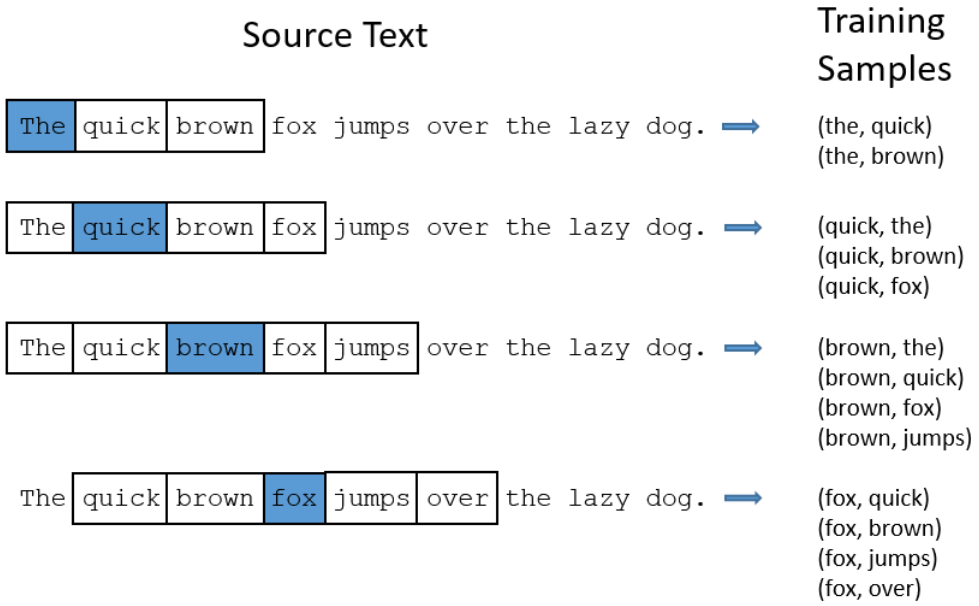
- Hypothèse de base :
- Des mots qui apparaissent dans un contexte similaire ont tendance à avoir une signification similaire



2013 - Word embeddings

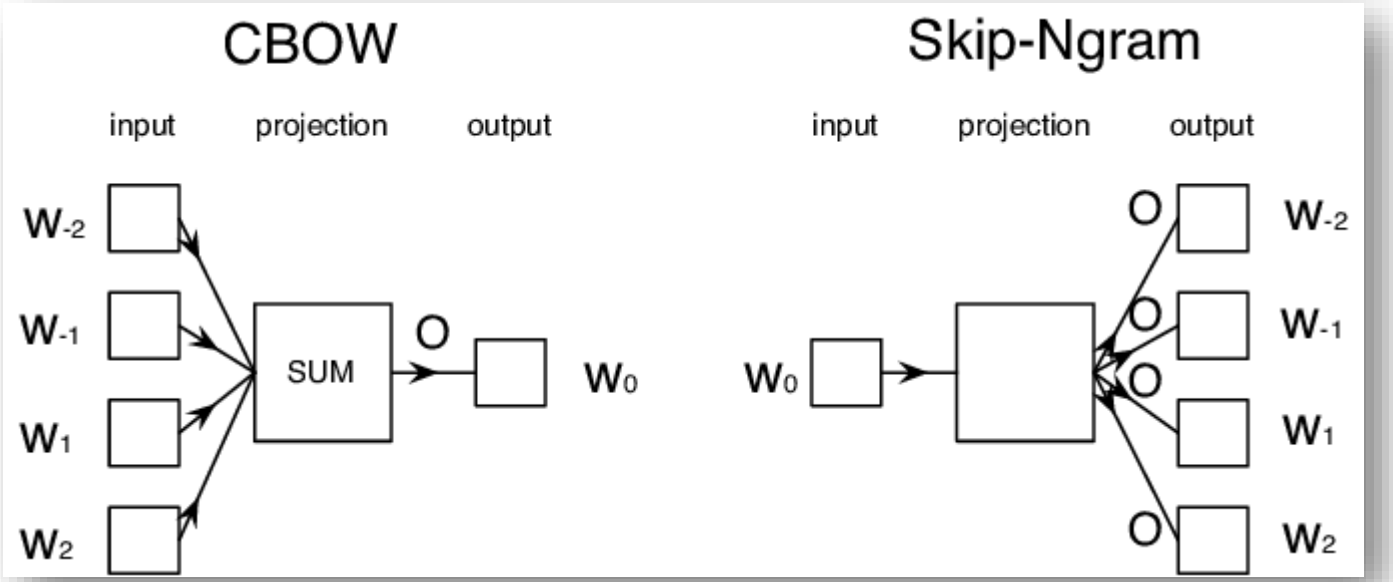
Idée de base

- Créer une tâche difficile
- Pour la résoudre le réseau doit apprendre une bonne représentation



- Par exemple : à partir d'un mot prédire son contexte !?
- => word2vec skip-gram

Word2vec

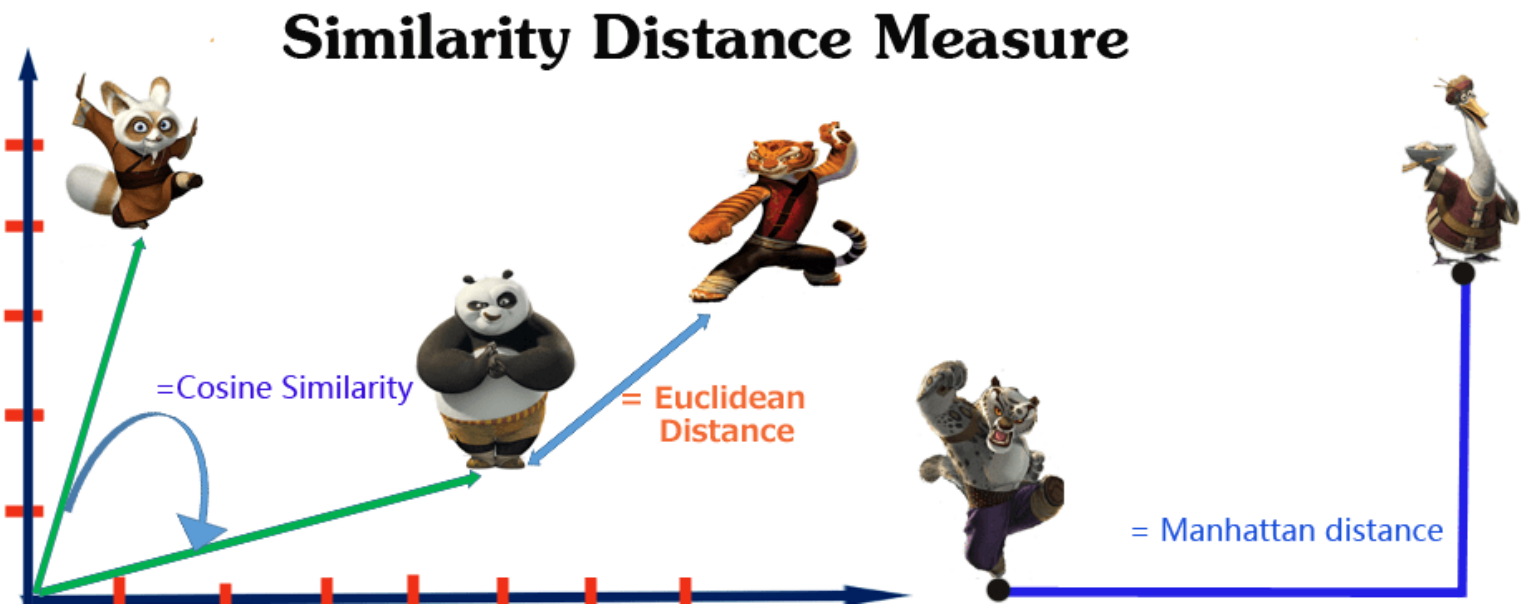


word2vec

“PS! Thank you for such an awesome top”

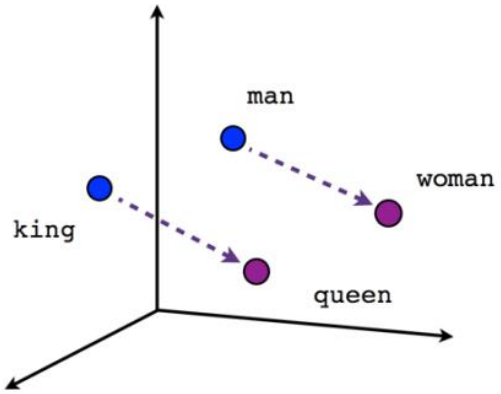
http://papers.nips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality.pdf

Similarité

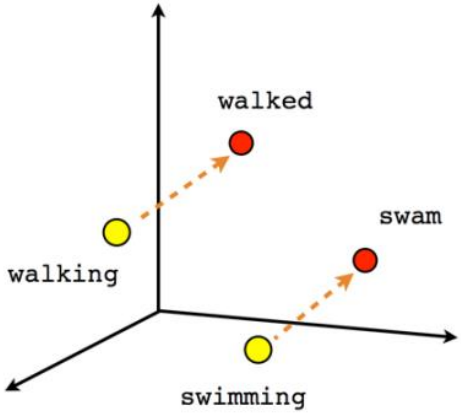


http://dataaspirant.com/2015/04/11/five-most-popular-similarity-measures-implementation-in-python/

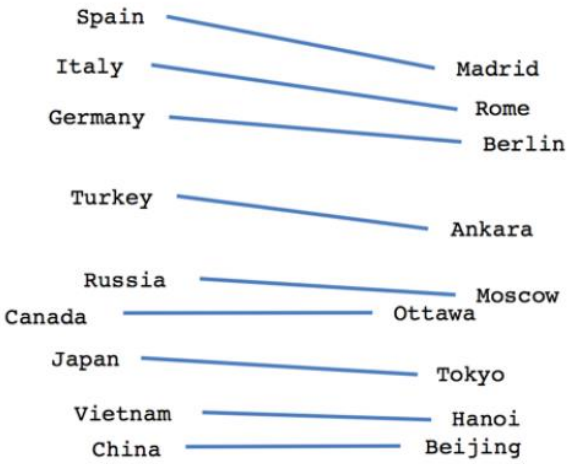
word2Vec



Male-Female



Verb tense



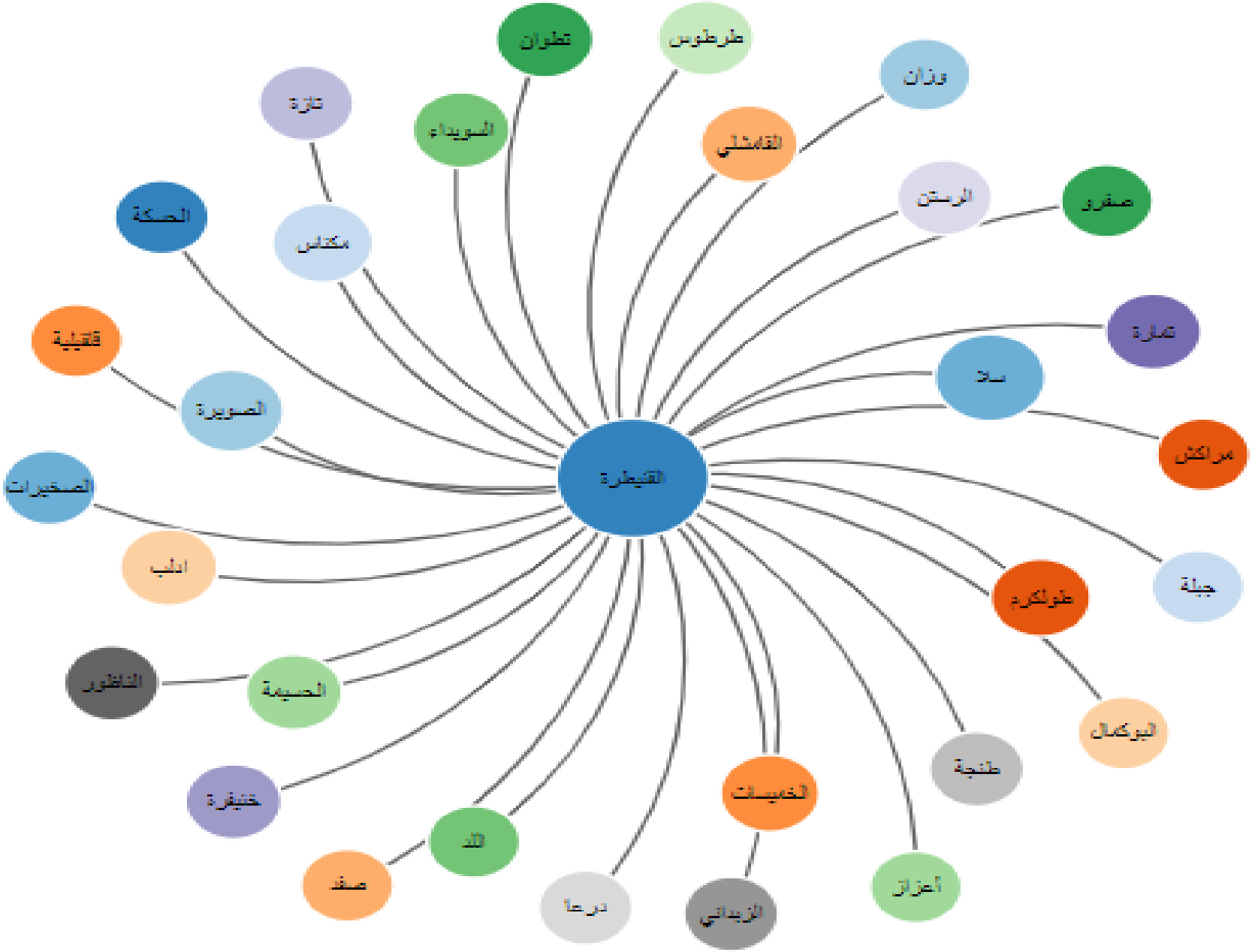
Country-Capital

opérations algébriques

Enter all three words, the first two, or the last two and see the words that result.

woman + (king - man) = Get result

king	0.8010287627819904
queen	0.5515625812235518
princess	0.4533671230317438


القنيطرة
تطوان
طرطوس
وزان
تازة
السويداء
القامشلي
الرستن
صفرو
الحسكة
مكناس
قلعية
سلا
تمارة
الصويرة
مراكش
الصخيرات
اللاذقية
جبلة
طرطوس
البوكمال
الناظور
الحسيمة
طنجة
الخميسات
خنيفرة
اللاذ
أعزاز
صفد
درعا
الزيداني

Bag-of-words : Limites

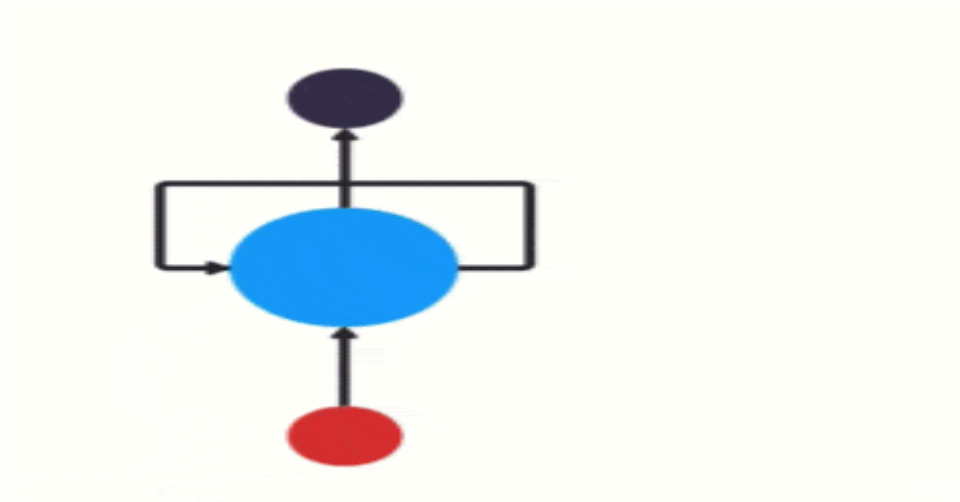
- Exemple
  - Je recommande ce produit qui n'est pas mauvais
  - Je ne recommande pas ce produit qui est pas mauvais

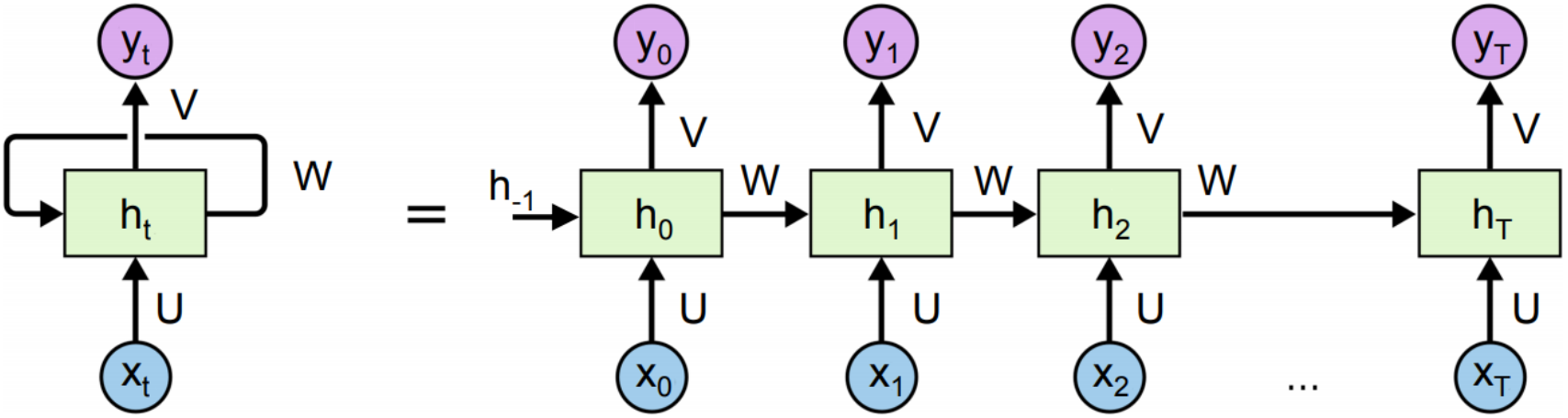
même vecteur

  - L'information temporelle très importante
- Un texte => une séquence temporelle de longueur variable

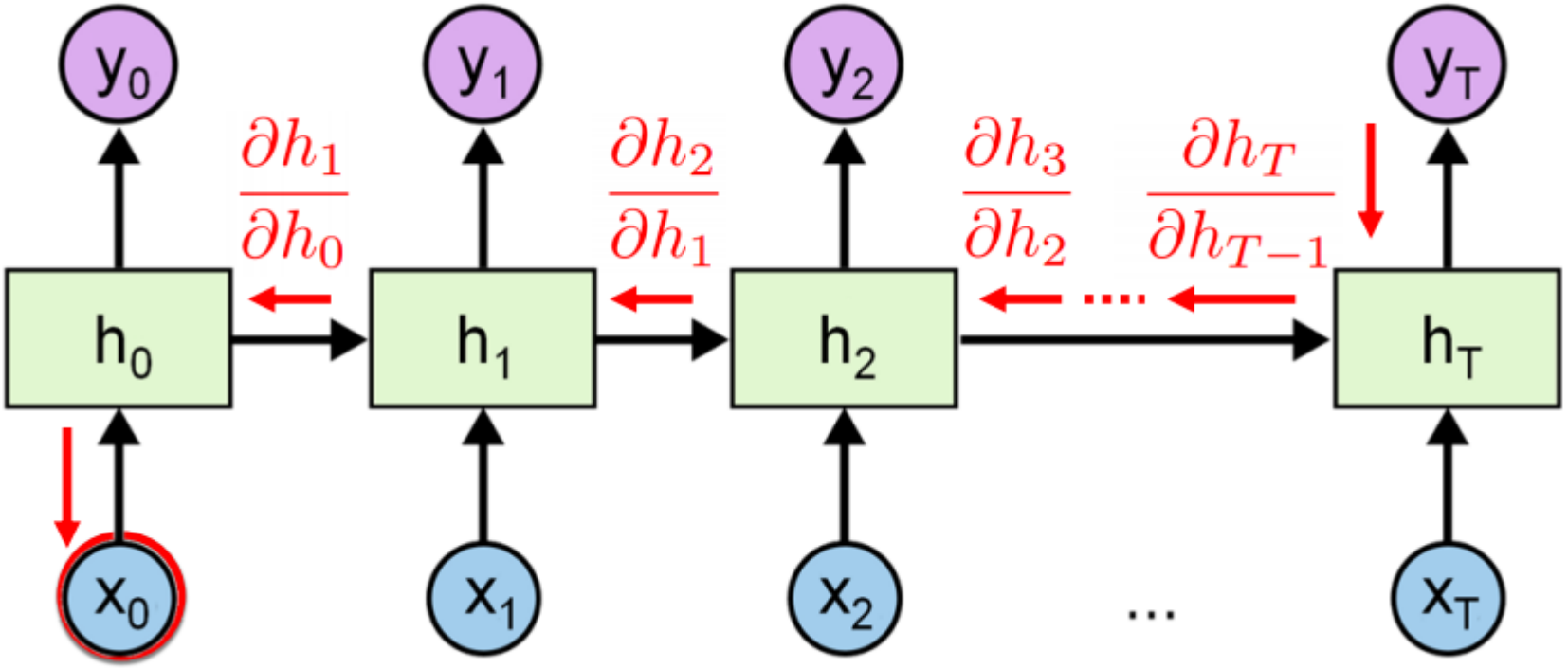
2013 - Neural networks for NLP

- **Recurrent neural networks**





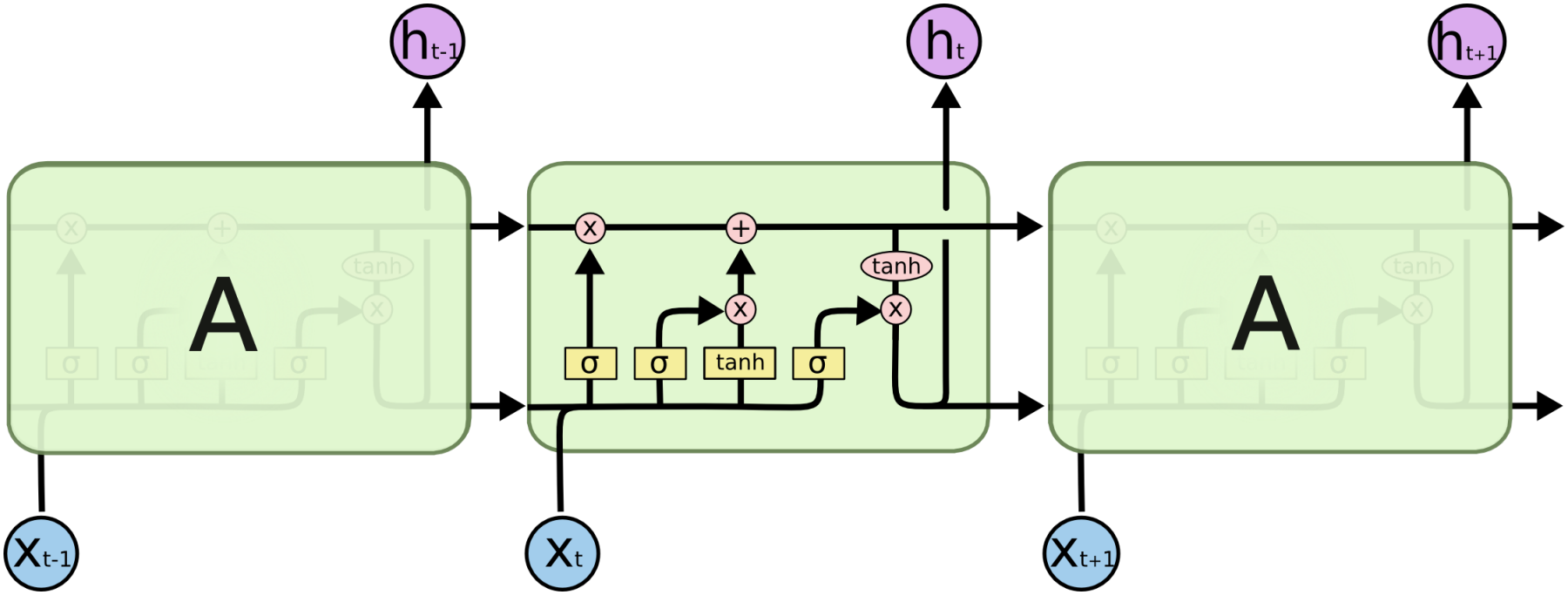
RNN



Apprendre des dépendances à long terme
= Propager le gradient loin dans le temps

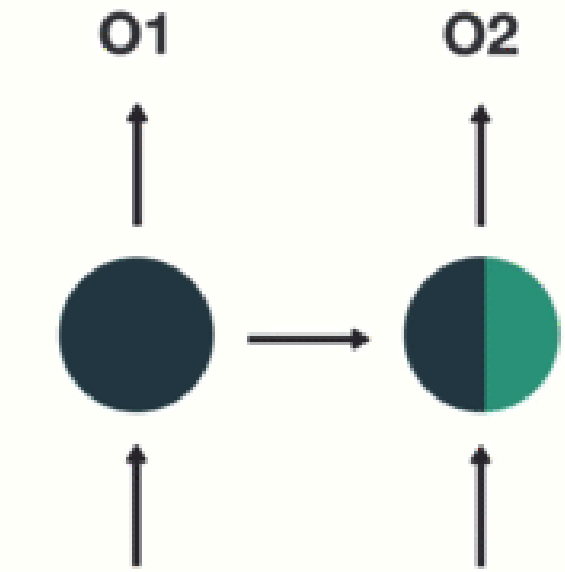
Pb. Mémoire

LSTM



Problème de séquences

Avec LSTM



What time is it ?

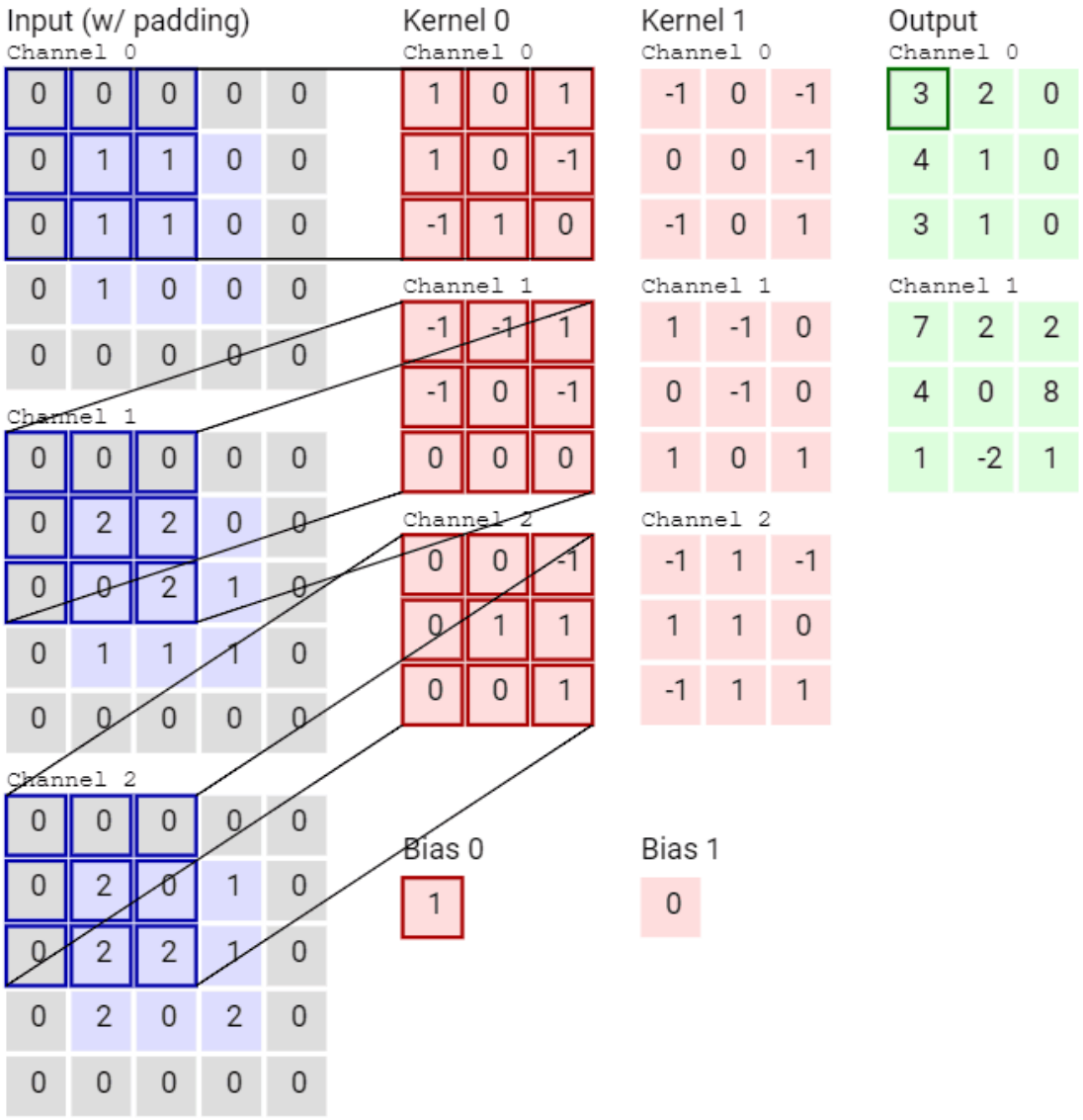
Avec LSTM



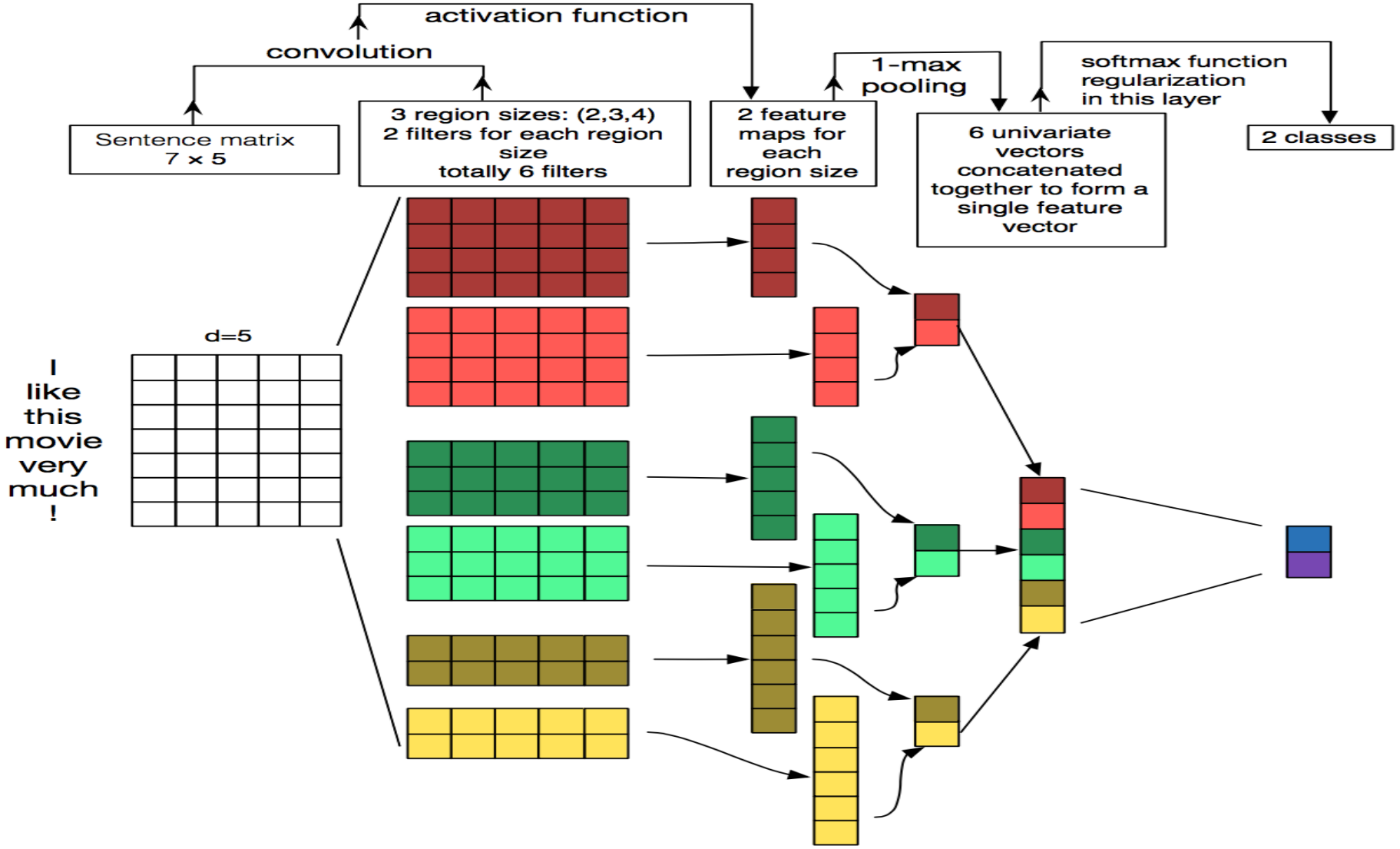
Lexical class:

JOHN: PERSON_BY_FIRSTNAME
HUMAN_BY_NAME
BEING_BY_NAME

CNN

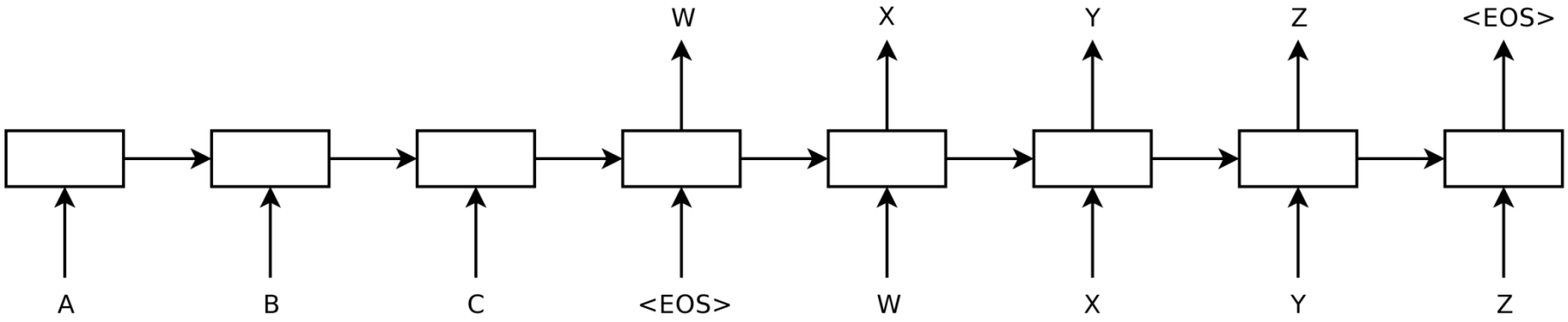

Input (w/ padding)
Channel 0
0 0 0 0 0
0 1 1 0 0
0 1 1 0 0
0 1 0 0 0
0 0 0 0 0
Channel 1
0 0 0 0 0
0 2 2 0 0
0 0 2 1 0
0 1 1 1 0
0 0 0 0 0
Channel 2
0 0 0 0 0
0 2 0 1 0
0 2 2 1 0
0 2 0 2 0
0 0 0 0 0
Kernel 0
Channel 0
1 0 1
1 0 -1
-1 1 0
Channel 1
-1 -1 1
-1 0 -1
0 0 0
Channel 2
0 0 -1
0 1 1
0 0 1
Bias 0
1
Kernel 1
Channel 0
-1 0 -1
0 0 -1
-1 0 1
Channel 1
1 -1 0
0 -1 0
1 0 1
Channel 2
-1 1 -1
1 1 0
-1 1 1
Bias 1
0
Output
Channel 0
3 2 0
4 1 0
3 1 0
Channel 1
7 2 2
4 0 8
1 -2 1

CNN


activation function
convolution
1-max pooling
softmax function regularization in this layer
Sentence matrix 7 × 5
3 region sizes: (2,3,4) 2 filters for each region size totally 6 filters
2 feature maps for each region size
6 univariate vectors concatenated together to form a single feature vector
2 classes
d=5
I
like
this
movie
very
much
!

2014 - Sequence-to-sequence models

- Réseau de neurones récurrent qui convertit une séquence de données d'un domaine en entrée vers une nouvelle séquence de données dans un autre domaine en sortie.
- Deux réseaux de neurones récurrents, un premier réseau est **un encodeur** et le second est **un décodeur**

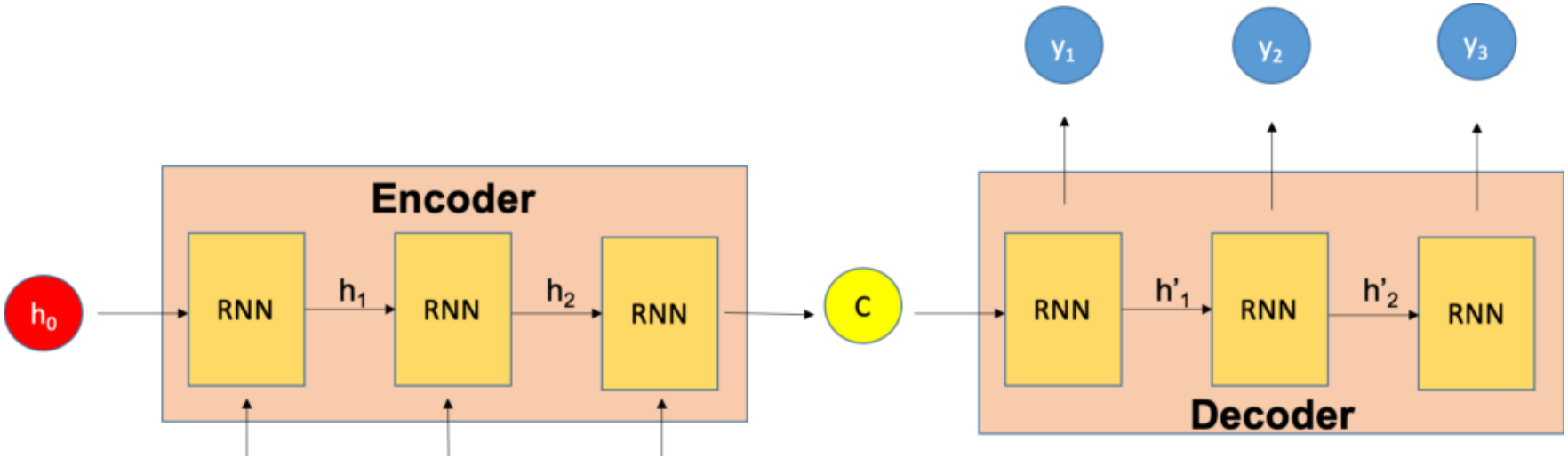


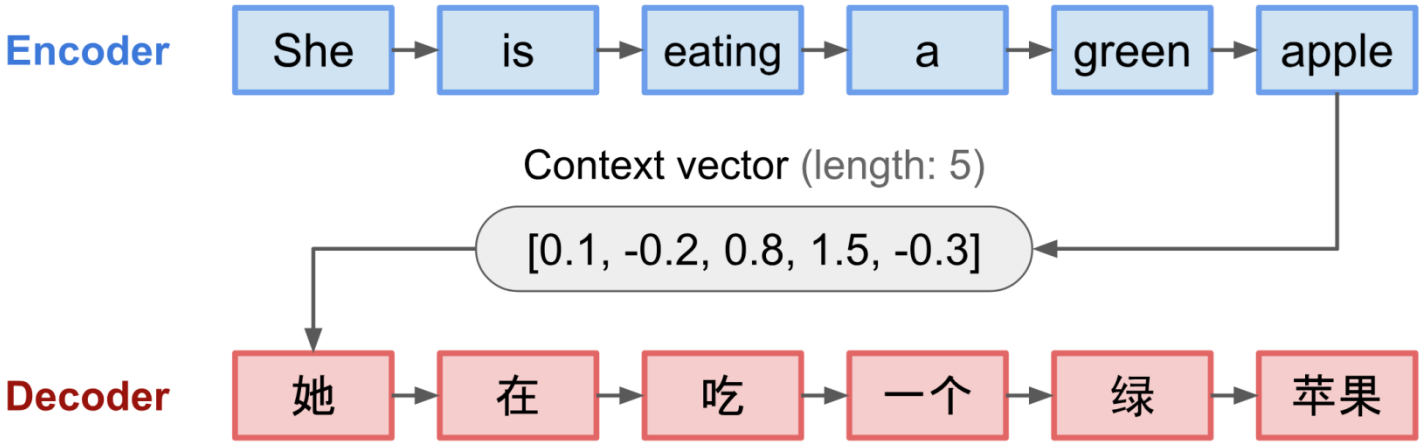
Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. In Advances in Neural Information Processing Systems

Seq2seq

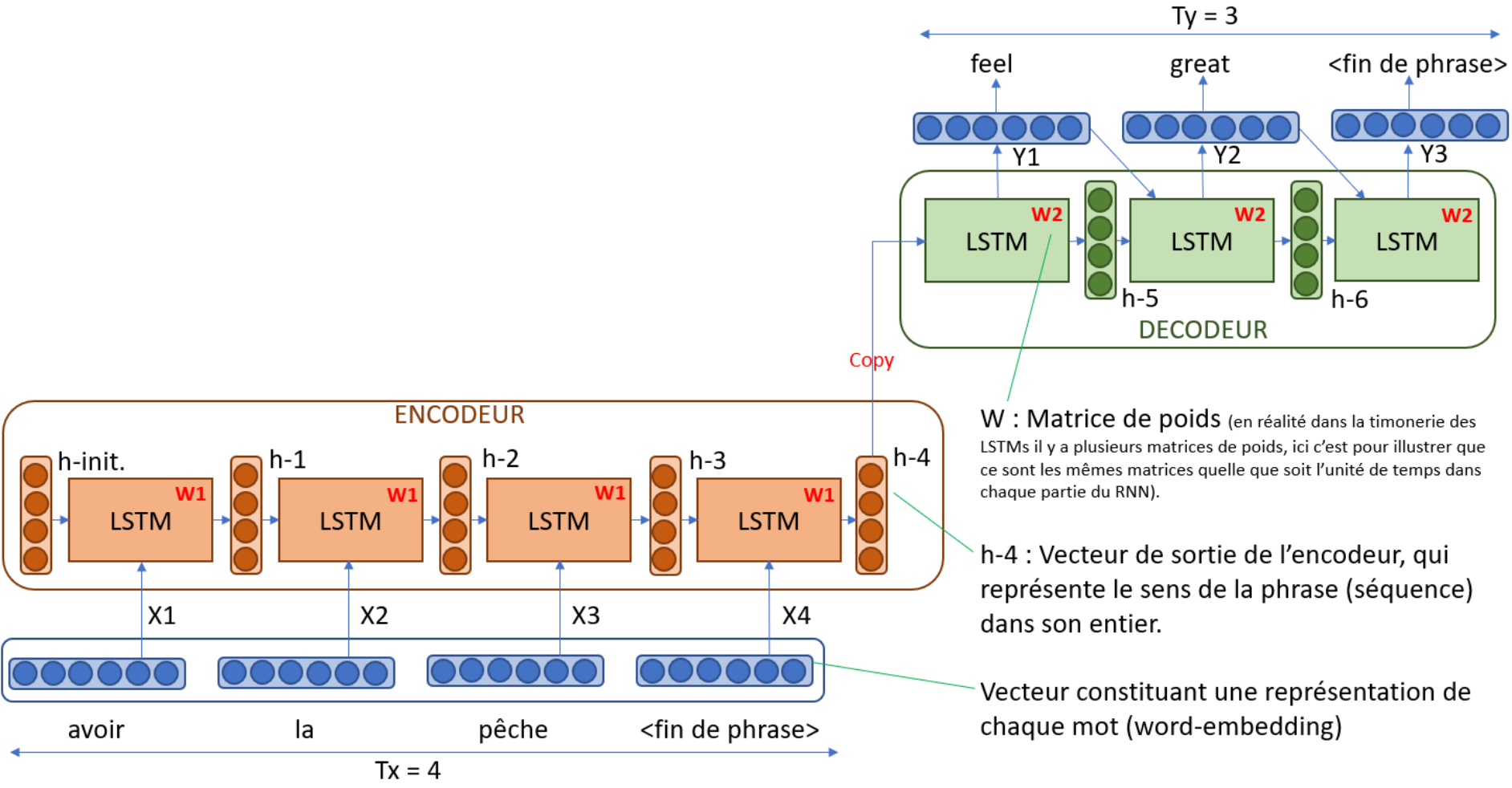
- Le modèle seq2seq a normalement une architecture codeur-décodeur, composée de:
- Un **encodeur** traite la séquence d'entrée et compresse les informations dans un vecteur de contexte (également connu sous le nom d'incorporation de phrases ou vecteur «pensé»/ sentence embedding or “thought” vector) d'une *longueur fixe* . Cette représentation devrait être un bon résumé de la signification de la séquence source *entière* .
- Un **décodeur** est initialisé avec le vecteur de contexte pour émettre la sortie transformée. Les premiers travaux utilisaient uniquement le dernier état du réseau de codeurs comme état initial du décodeur.
- L'encodeur et le décodeur sont tous deux des réseaux de neurones récurrents, c'est-à-dire utilisant des unités LSTM ou GRU .

Encodeur – Décodeur





Encodeur – Décodeur (exemple)



W : Matrice de poids (en réalité dans la timonerie des LSTMs il y a plusieurs matrices de poids, ici c'est pour illustrer que ce sont les mêmes matrices quelle que soit l'unité de temps dans chaque partie du RNN).

h-4 : Vecteur de sortie de l'encodeur, qui représente le sens de la phrase (séquence) dans son entier.

Vecteur constituant une représentation de chaque mot (word-embedding)

Pourquoi ?

- Une critique et un inconvénient et apparent de cette conception de vecteur de contexte de longueur fixe est l'incapacité de se souvenir des phrases longue.
- Souvent, il a oublié la première partie une fois qu'il a terminé le traitement de la totalité de l'entrée.
- Le mécanisme d'attention est né (Bahdanau et al., 2015) pour résoudre ce problème.

Attention



A dog is standing on a hardwood floor.



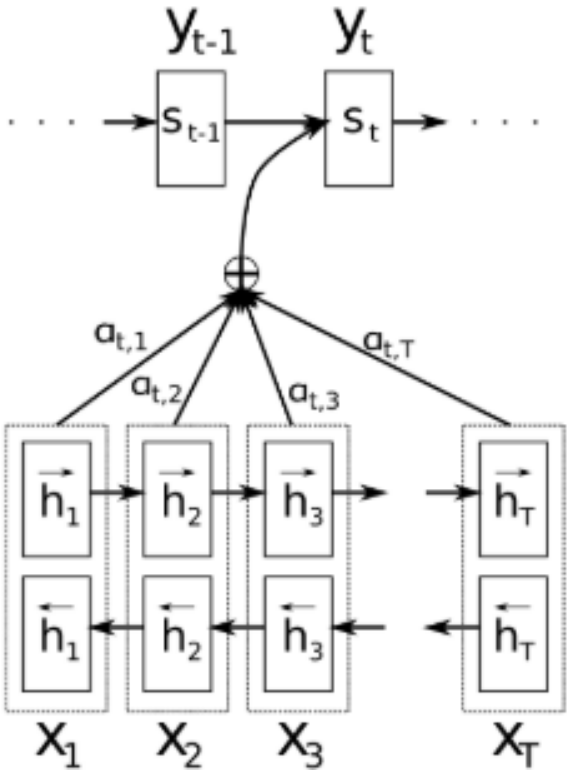
A stop sign is on a road with a mountain in the background.



A woman is throwing a frisbee in a park

2015 - Attention

- Un mécanisme d'attention permet à un RNN de porter son attention sur différentes parties de son entrée [x0 , …, xT].
  1. Chaque élément de l'entrée est comparé à l'état courant ht-1, pour générer des coefficients de pondération **αi** .
  2. La sortie ct (contexte) est une somme pondérée de tous les éléments de l'entrée (« *Soft Attention* »).



1. *score* : $\alpha_i^j = a(z^i, h_j)$,

2. *attention score* : $\hat{\alpha}_i^j = \frac{e^{\alpha_i^j}}{\sum_{k=1}^{n} e^{\alpha_i^k}}$

Bahdanau, D., Cho, K., & Bengio,Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. In ICLR 2015.

Attention

- Soit une séquence source X de longueur n et essayez de sortir une séquence cible $\boldsymbol{Y}$ de longueur m:

$$\mathbf{x} = [x_1, x_2, \ldots, x_n]$$
$$\mathbf{y} = [y_1, y_2, \ldots, y_m]$$

- L'encodeur est un RNN bidirectionnel (ou tout autre type de réseau récurrent) avec un état caché vers l'avant $\overrightarrow{\boldsymbol{h}}_i$ et un $\overleftarrow{\boldsymbol{h}}_i$ en arrière. Une concaténation simple de deux représente l'état du codeur. La motivation est d'inclure les mots précédents et suivants dans l'annotation d'un mot.

$$\boldsymbol{h}_i = [\overrightarrow{\boldsymbol{h}}_i^\top ; \overleftarrow{\boldsymbol{h}}_i^\top]^\top, i = 1, \ldots, n$$

Formalisme

- Le réseau du décodeur a un état caché $\boldsymbol{s}_t = f(\boldsymbol{s}_{t-1}, y_{t-1}, \mathbf{c}_t)$ pour le mot de sortie à la position t, t=1,...,m

où le vecteur de contexte ***ct*** est un somme d'états masqués de la séquence d'entrée, pondérée par les scores d'alignement:

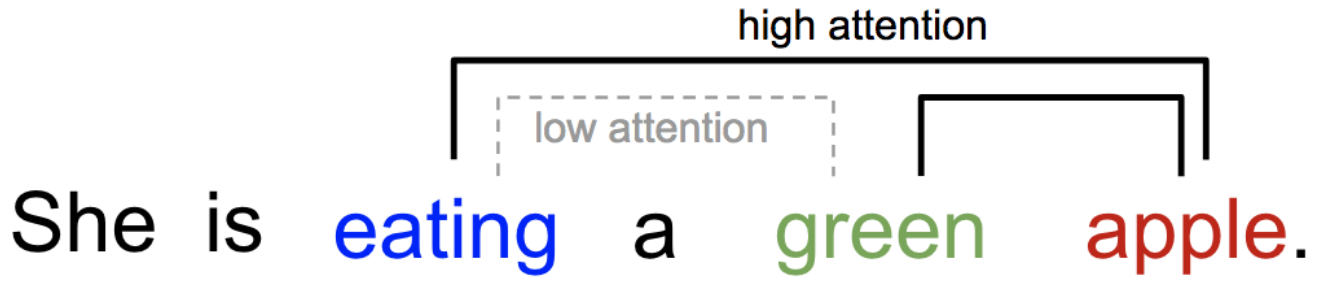
$$\mathbf{c}_t = \sum_{i=1}^{n} \alpha_{t,i} \boldsymbol{h}_i \quad ; \text{Context vector for output } y_t$$

$$\alpha_{t,i} = \text{align}(y_t, x_i) \quad ; \text{How well two words } y_t \text{ and } x_i \text{ are aligned.}$$

$$= \frac{\exp(\text{score}(\boldsymbol{s}_{t-1}, \boldsymbol{h}_i))}{\sum_{i'=1}^{n} \exp(\text{score}(\boldsymbol{s}_{t-1}, \boldsymbol{h}_{i'}))} \quad ; \text{Softmax of some predefined alignment score..}$$

Attention

- l'attention dans le deep learning peut être largement interprétée comme **un vecteur de poids d'importance**: afin de prédire ou de déduire un élément, tel qu'un pixel dans une image ou un mot dans une phrase, nous estimons à l'aide du vecteur d'attention comment il est fortement corrélé avec (ou « *assiste à* » comme vous avez pu le lire dans de nombreux articles) d'autres éléments et prend la somme de leurs valeurs pondérées par le vecteur d'attention comme approximation de la cible.



One word "attends" to other words in the same sentence differently.

Attention

1) **Global attention**: Uses all hidden states from the encoder to compute the context vector. This is computationally costly as all words from source are considered for each target word.

2) **Local attention**: Chooses a position in the source sentence to determine the window of words to consider

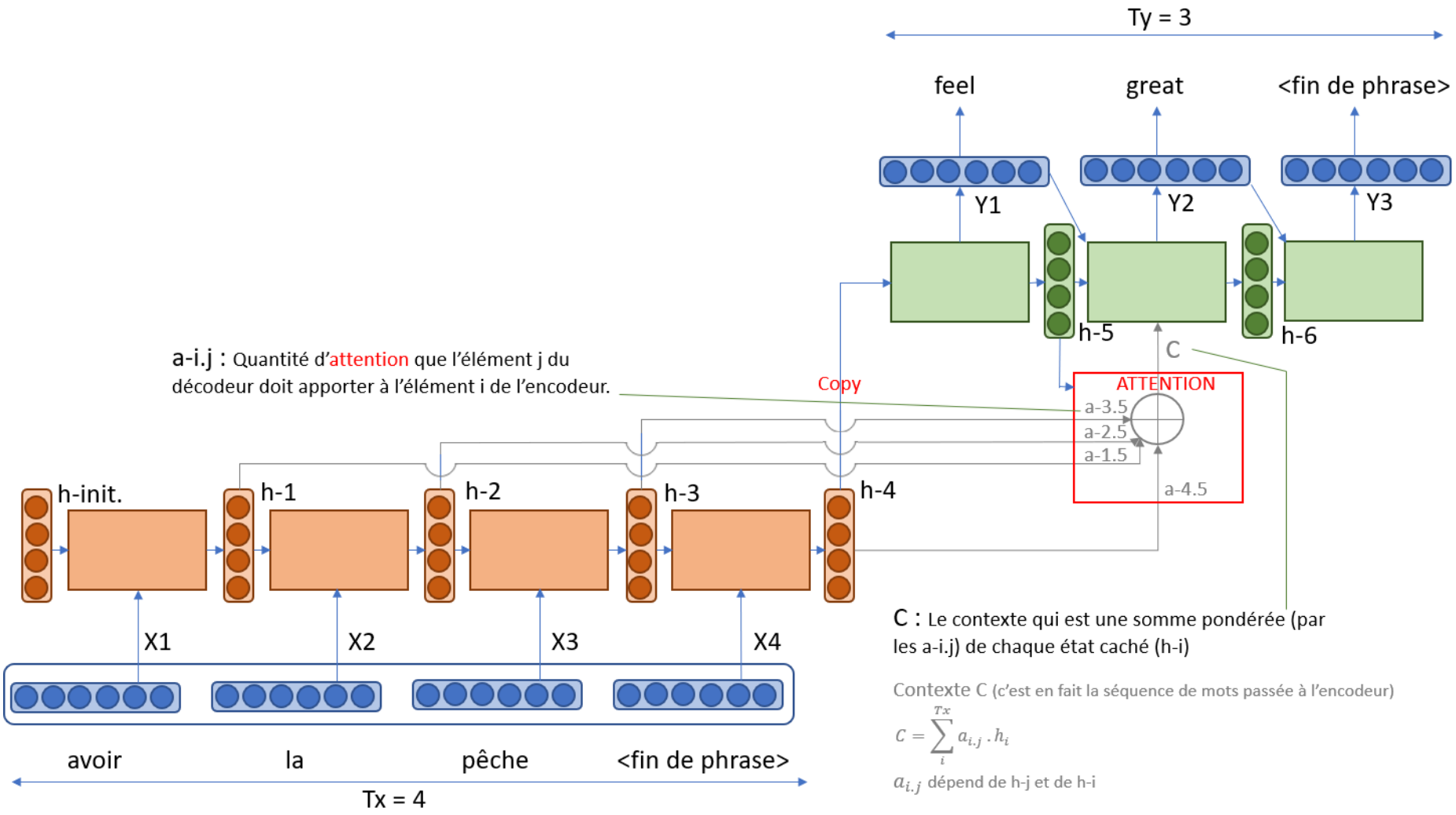
3) **Two-way attention**: The same model attends to the hypothesis and premise, and both representations are concatenated. However, this model can't differentiate that the alignment between stop words is less important than alignment between content words.

4) **Self-attention**: A mechanism relating different positions of a single sequence to compute its internal representation.

5) **Key-value attention**: Output vectors are separated into keys to calculate attention and values to encode the next-word distribution and context representation.

6) **Hierarchical-nested attention**: Two attention levels - first at the word level and second at the sentence level. This highlights the highly informative components of a document.

Attention


Ty = 3
feel
great
<fin de phrase>
Y1
Y2
Y3
h-5
h-6
C
ATTENTION
a-3.5
a-2.5
a-1.5
a-4.5
Copy
a-i.j : Quantité d'attention que l'élément j du décodeur doit apporter à l'élément i de l'encodeur.
h-init.
h-1
h-2
h-3
h-4
X1
X2
X3
X4
avoir
la
pêche
<fin de phrase>
Tx = 4
C : Le contexte qui est une somme pondérée (par les a-i.j) de chaque état caché (h-i)
Contexte C (c'est en fait la séquence de mots passée à l'encodeur)
$C = \sum_{i}^{Tx} a_{i.j} \, . \, h_i$
$a_{i.j}$ dépend de h-j et de h-i

a_{ij}

- Comment peuvent être calculés les *aij*. ?
- (Il en existe pas mal) trois grandes familles
- L'attention stricte (hard attention), qui focalise sur un seul élément du contexte et qui est stochastique (pas de notion de dérivée, la sélection de l'élément "élu" et la rétro-propagation se font via des méthodes statistiques, échantillonnages, distributions)
- L'attention "molle" (soft attention), la plus classique.
- L'attention locale qui ne sélectionne que quelques éléments proches sur lesquels porter l'attention. C'est un hybride entre la soft et la hard attention.

Soft attention

Les h-i sont les sorties cachées/activations de l'encodeur, et les h-j celles du décodeur

$$m_{ij} = \tanh(W_1 . h_i + W_2 . h_{(j-1)})$$

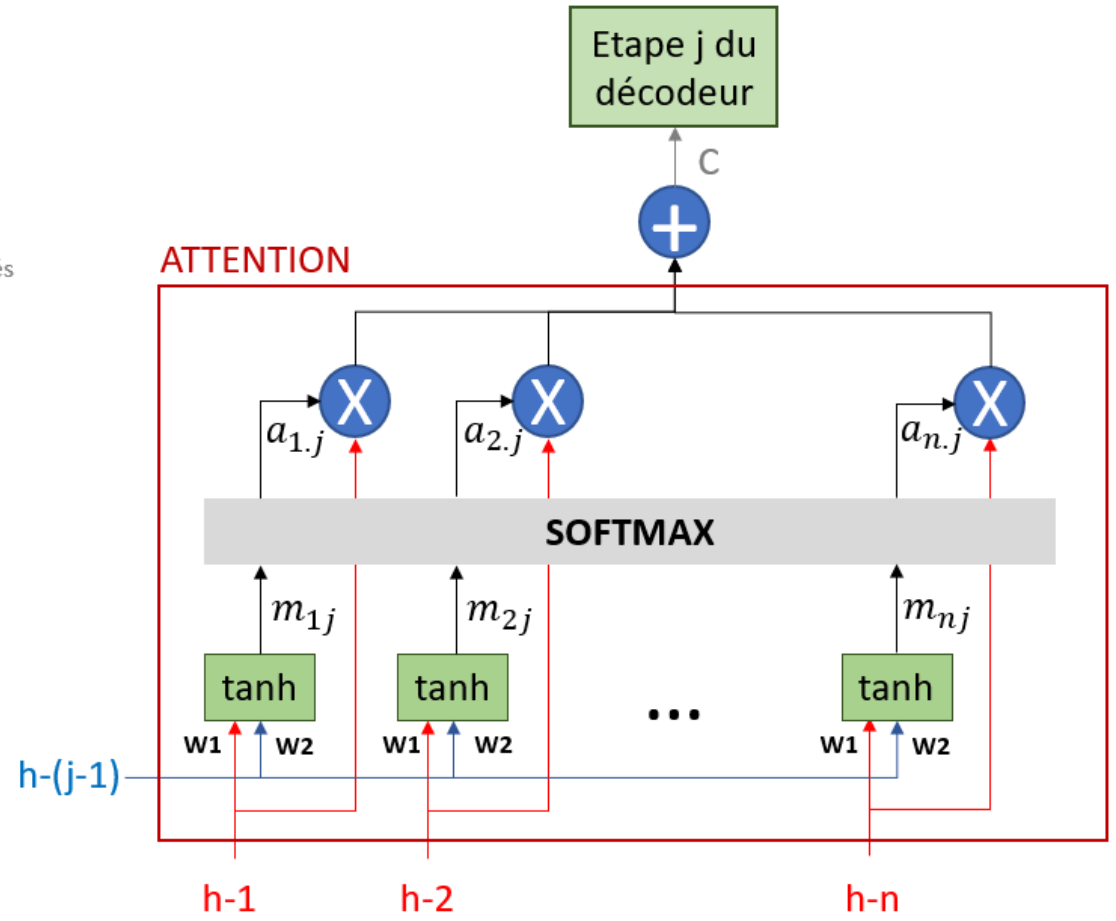
(W1 et W2 peuvent être entrainées, leurs poids sont recalculés par rétropropagation)

La softmax va ensuite donner en sortie une somme de poids à 1

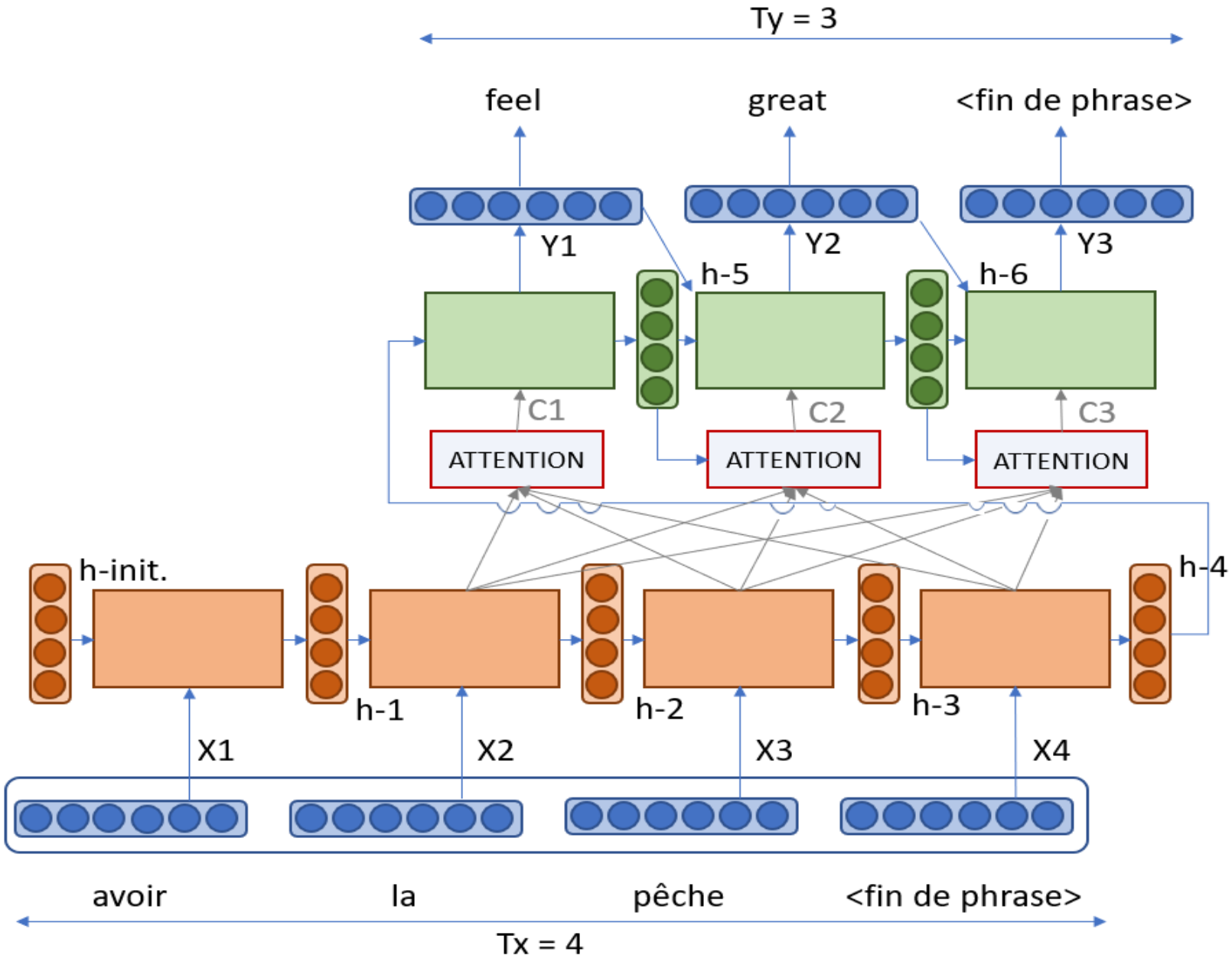
$$a_{ij} = \frac{m_{ij}}{\sum_k^{Tx} \exp(m_{i.k})}$$

Et les a_{ij} somment à 1.
Puis enfin

$$C = \sum_i^{Tx} a_{i.j} \, . h_i$$

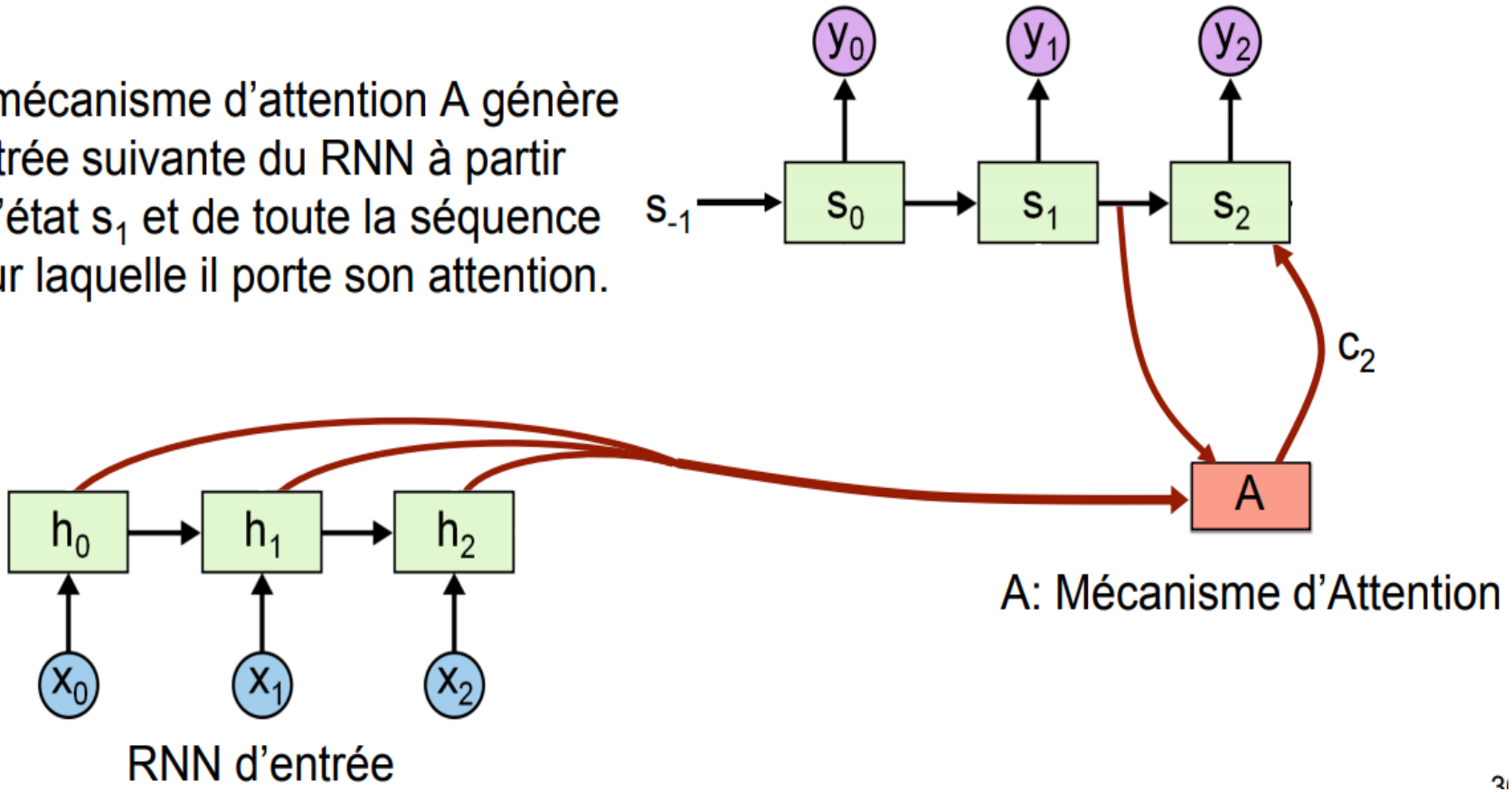


https://lesdieuxducode.com/blog/2019/4/bert--le-transformer-model-qui-sentraine-et-qui-represente

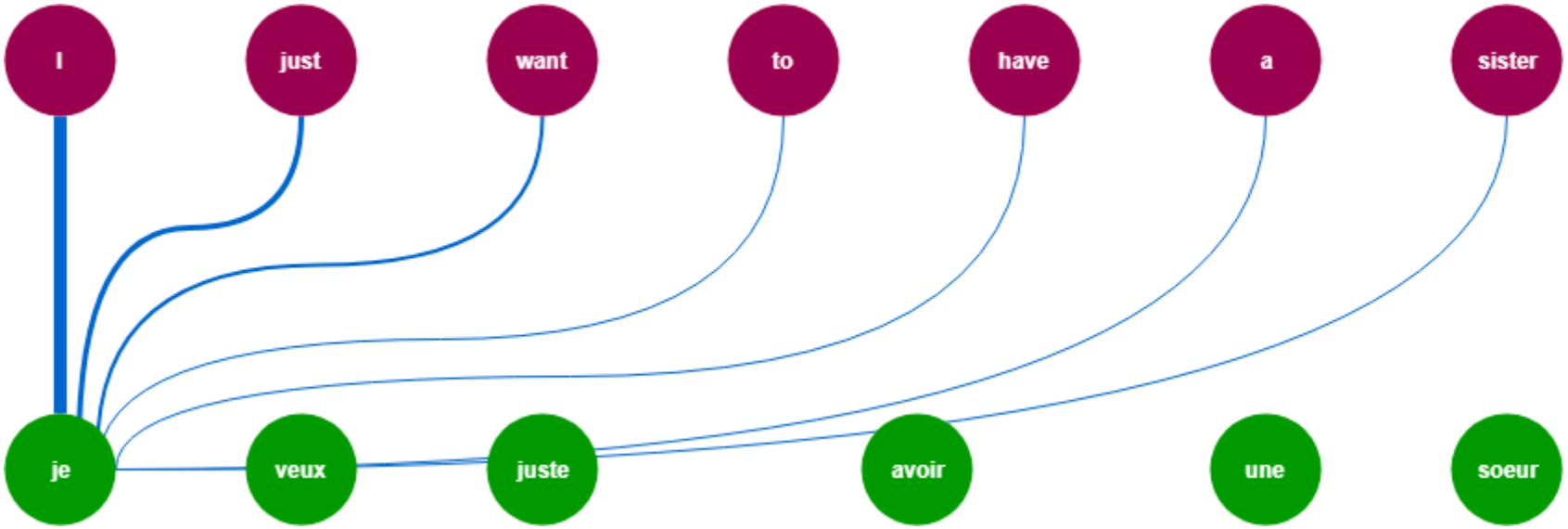

Ty = 3
feel
great
<fin de phrase>
Y1
Y2
Y3
h-5
h-6
C1
C2
C3
ATTENTION
ATTENTION
ATTENTION
h-init.
h-4
h-1
h-2
h-3
X1
X2
X3
X4
avoir
la
pêche
<fin de phrase>
Tx = 4

Attention

Le mécanisme d'attention A génère l'entrée suivante du RNN à partir de l'état s_1 et de toute la séquence h sur laquelle il porte son attention.



Attention

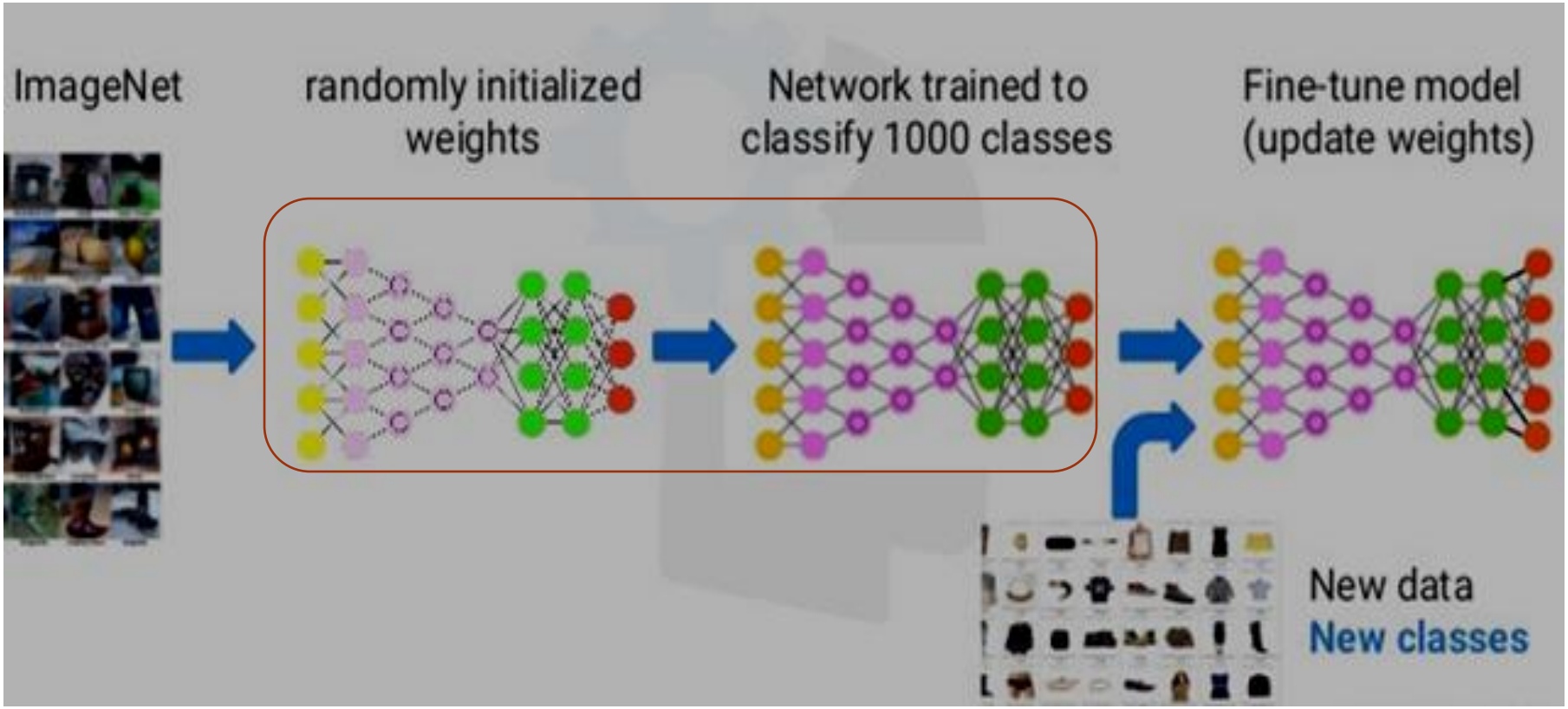


- Les traits blues en gras montrent où le RNN mauve (décodeur) porte son attention dans la séquence générée par le RNN vert (encodeur).

Modèle profond pré-entraîné : la révolution de 2018

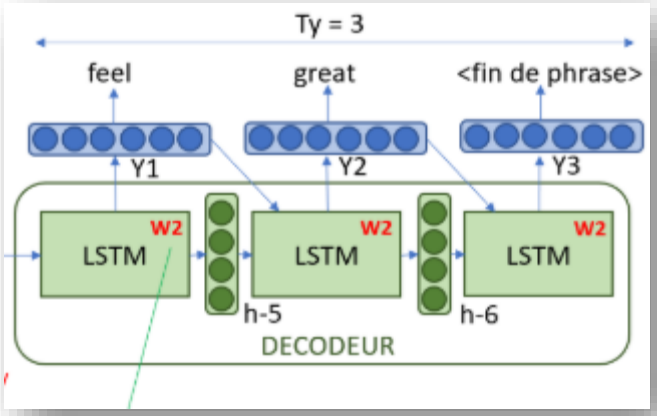
- Nouvelle approche est similaire à celle utilisée dans le domaine de la vision par ordinateur
- Entraîner un modèle complet à réaliser une tâche avec **beaucoup de données**.
- Puis, se servir de ce modèle pré-entraîné pour réaliser d'autres tâches, en bénéficiant de l'apprentissage précédent.
- C'est ce que l'on appelle le **transfert d'apprentissage**.

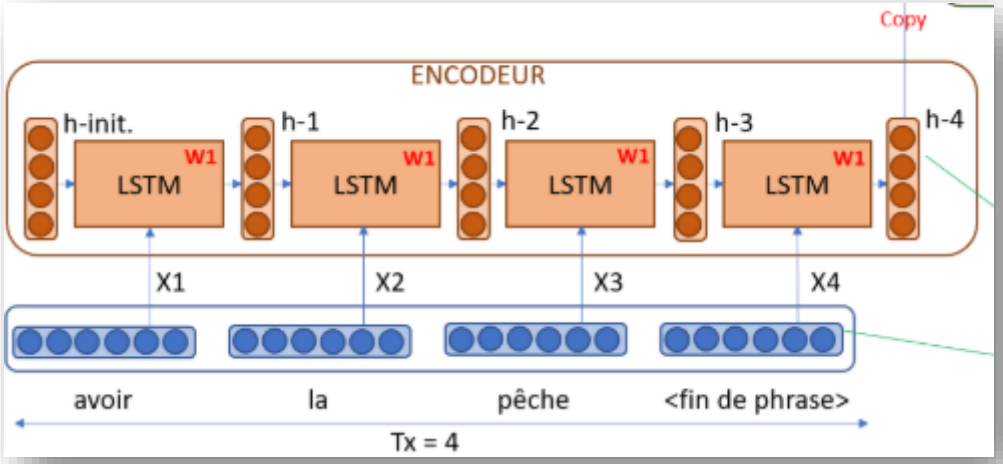
Transfer Learning



self-attention

- **Les mécanismes d'auto-attention** (self-attention) sont similaires, sauf qu'**au lieu de s'opérer entre les éléments de l'encodeur et du décodeur**, ils **s'opèrent sur les éléments de l'input entre eux** (le présent regarde le passé et le futur) **et de l'output entre eux** aussi (le présent regarde le passé, vu que le futur est encore à générer).

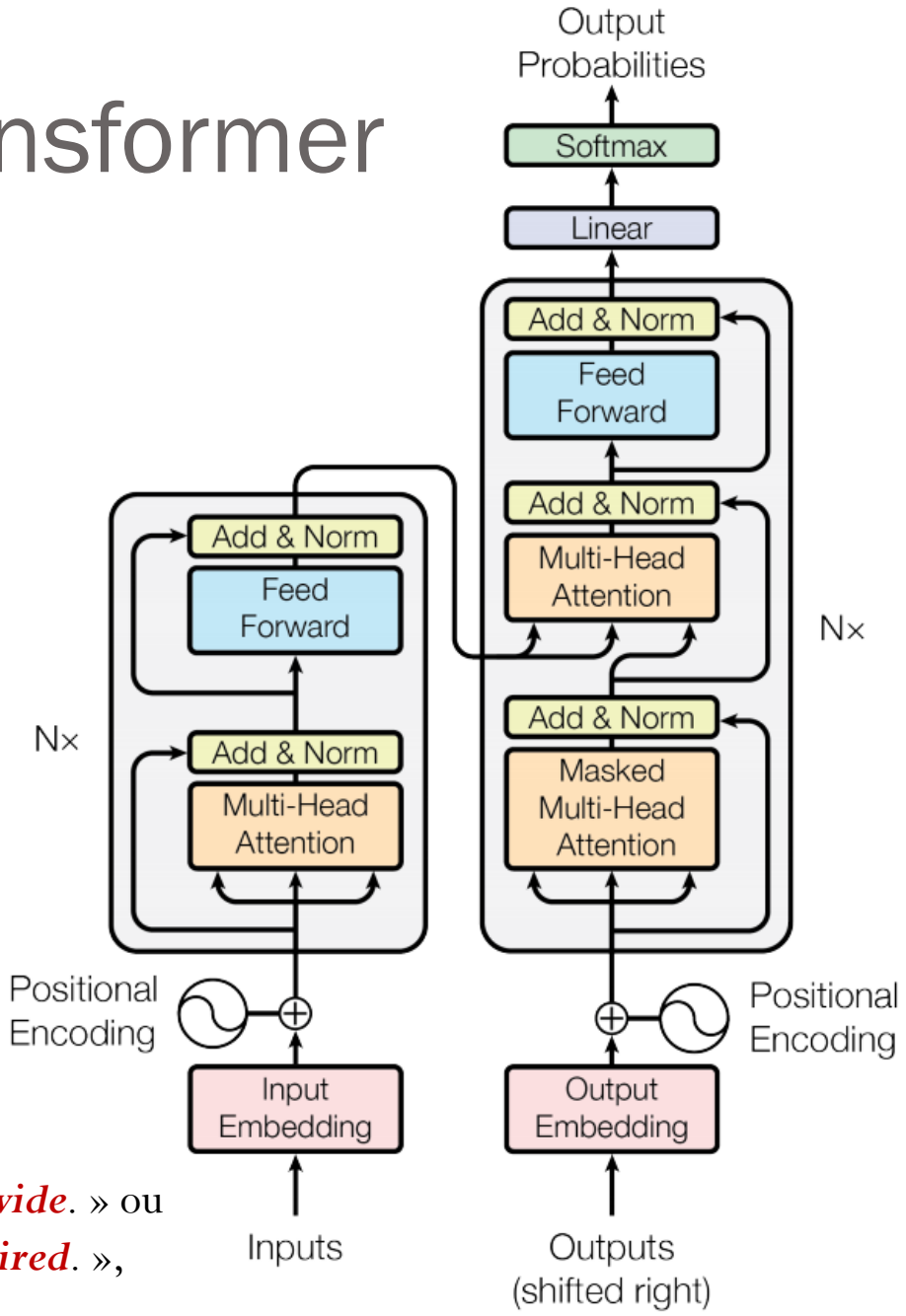




L'architecture Transformer

« ***Attention is All You Need !*** »

La motivation des concepteurs du Transformer était de pallier un des principaux inconvénients des RNN à savoir l'impossibilité de **paralléliser** leurs traitements en raison du caractère **séquentiel** des calculs qu'ils effectuent.



« *The animal did not cross the* ***street*** *because* ***it*** *was* ***wide***. » ou
« *The* ***animal*** *did not cross the street because* ***it*** *was* ***tired***. »,

Modèle profond pré-entraîné

- **ULMFiT** en janvier 2018.
- **ELMo** en février 2018.
- **GPT-2** d'OpenAI en juin 2018.
- **BERT** de Google en octobre 2018.

Algorithmes basés sur BERT (les plus connus)

- **RoBERTa** par Facebook,
- **MT-DNN** par Microsoft
- **XLNet** and **ALBERT** par Google et Toyota. Sorti en septembre 2019
- **BERT-mtl** par IBM
- **Google T5** par Google
- **DistilBERT** est une version plus petite, légère et rapide de BERT
- **FastBERT** est une version plus rapide de BERT
- **CamemBERT** est une version française développée Facebook AI, Inria et Université de Sorbonne 2019 (trained on 138GB of French text), dérivée de RoBERTa

BERT

BERT c'est pour **B**idirectional **E**ncoder **R**epresentations from **T**ransformers.

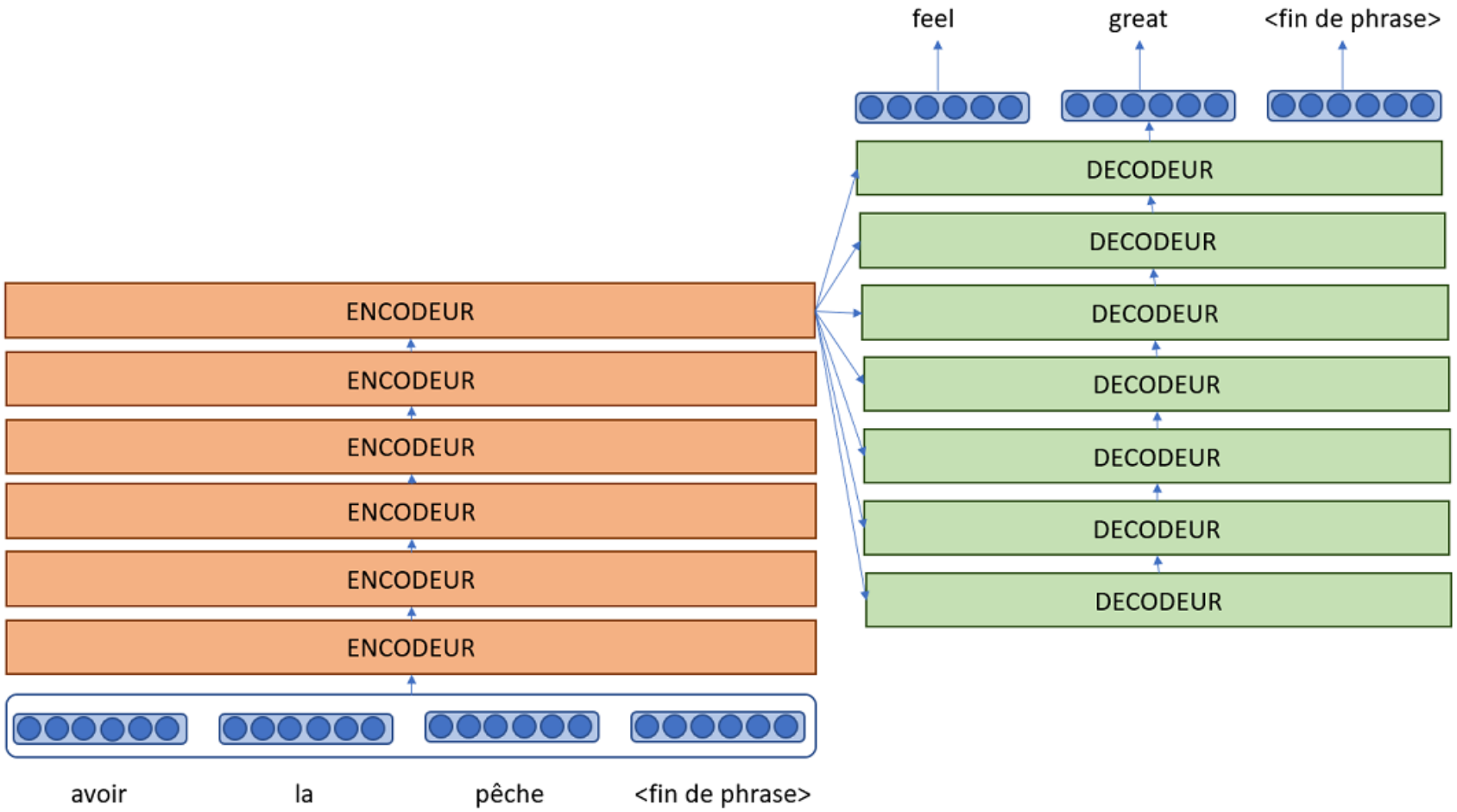
L'utilisation de **deux nouvelles tâches pour le pré-entraînement**, l'une au niveau des mots et l'autre au niveau des phrases.

- `BERT-Large, Uncased (Whole Word Masking)` : 24-layer, 1024-hidden, 16-heads, 340M parameters
- `BERT-Large, Cased (Whole Word Masking)` : 24-layer, 1024-hidden, 16-heads, 340M parameters
- `BERT-Base, Uncased` : 12-layer, 768-hidden, 12-heads, 110M parameters
- `BERT-Large, Uncased` : 24-layer, 1024-hidden, 16-heads, 340M parameters
- `BERT-Base, Cased` : 12-layer, 768-hidden, 12-heads , 110M parameters
- `BERT-Large, Cased` : 24-layer, 1024-hidden, 16-heads, 340M parameters
- `BERT-Base, Multilingual Cased (New, recommended)` : 104 languages, 12-layer, 768-hidden, 12-heads, 110M parameters
- `BERT-Base, Multilingual Uncased (Orig, not recommended)` **(Not recommended, use** `Multilingual Cased` **instead)**: 102 languages, 12-layer, 768-hidden, 12-heads, 110M parameters
- `BERT-Base, Chinese` : Chinese Simplified and Traditional, 12-layer, 768-hidden, 12-heads, 110M parameters

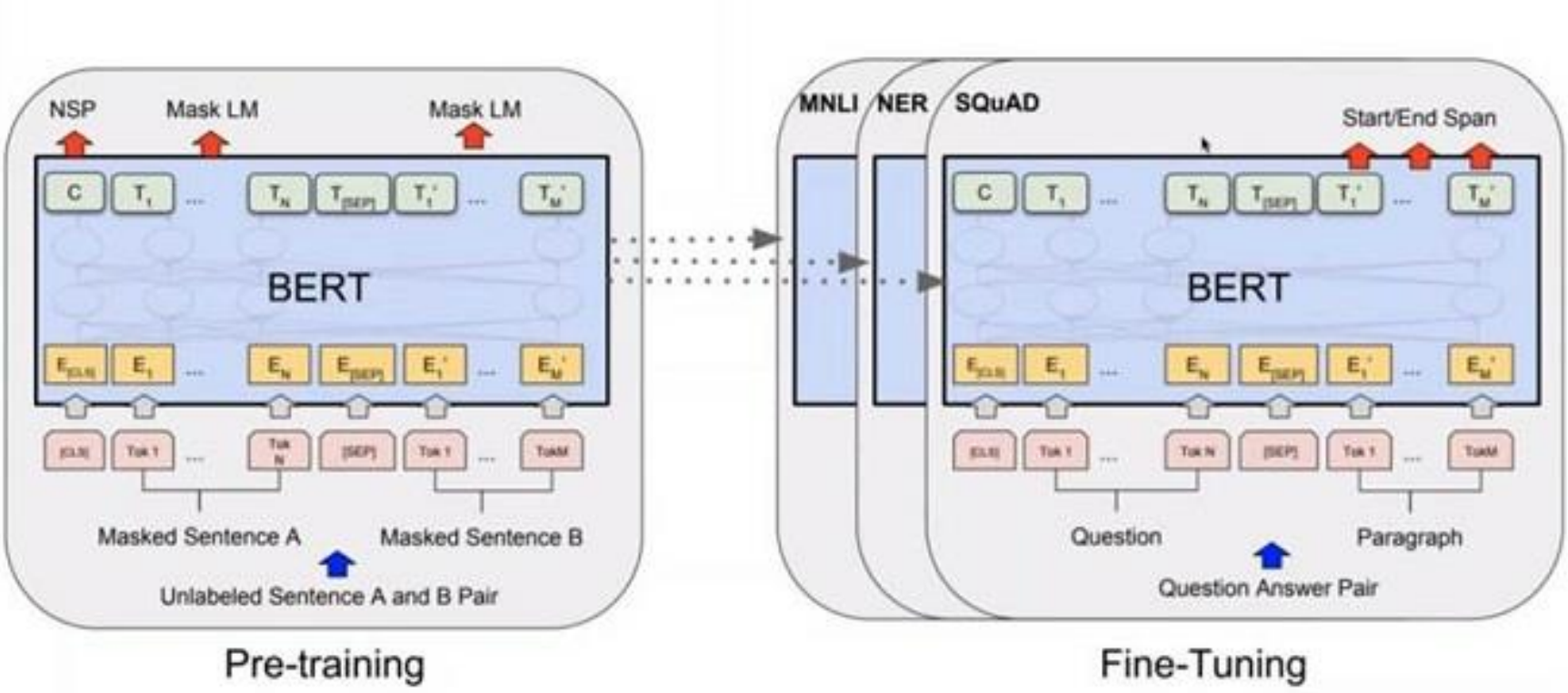
BERT

- BERT apprend de façon non supervisée
- le modèle va apprendre à prédire ce mot masqué.
  - Entrée = [CLS] the man went to [MASK] store [SEP]
  - Entrée = [CLS] the man [MASK] to the store [SEP]
- le modèle doit déterminer si la séquence suivante (suivant la séparation[SEP]) est bien la séquence suivante.
  - Entrée = [CLS] the man went to [MASK] store [SEP]
    he bought a gallon [MASK] milk [SEP]
    Label = IsNext
  - Entrée = [CLS] the man [MASK] to the store [SEP]
    penguin [MASK] are flight ##less birds [SEP]
    Label = NotNext

BERT


feel
great
<fin de phrase>
DECODEUR
DECODEUR
DECODEUR
DECODEUR
DECODEUR
DECODEUR
ENCODEUR
ENCODEUR
ENCODEUR
ENCODEUR
ENCODEUR
ENCODEUR
avoir
la
pêche
<fin de phrase>

BERT


NSP
Mask LM
Mask LM
BERT
Masked Sentence A
Masked Sentence B
Unlabeled Sentence A and B Pair
Pre-training
MNLI
NER
SQuAD
Start/End Span
BERT
Question
Paragraph
Question Answer Pair
Fine-Tuning

Conclusion

- Est-il possible d'apprendre une langue entièrement à partir d'un corpus de texte

- « Grounded Language Learning »(Y. Bengio)
  - apprendre un langage par **expérience sensorielle** plutôt que par la seule lecture….

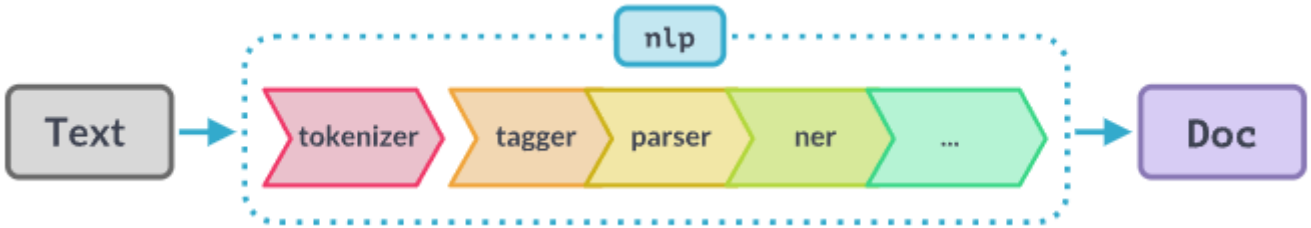
Part 2:

- Outils
- Bonne pratique

Outils

spacy

- conda install -c conda-forge spacy
- python -m spacy download en
- import spacy
  nlp = spacy.load('en_core_web_sm')



Tag	Description
.text	The original word text
.lemma_	The base form of the word
.pos_	The simple part-of-speech tag
.tag_	The detailed part-of-speech tag
.shape_	The word shape – capitalization, punctuation, digits
.is_alpha	Is the token an alpha character?
.is_stop	Is the token part of a stop list, i.e. the most common words of the language?

Exemple

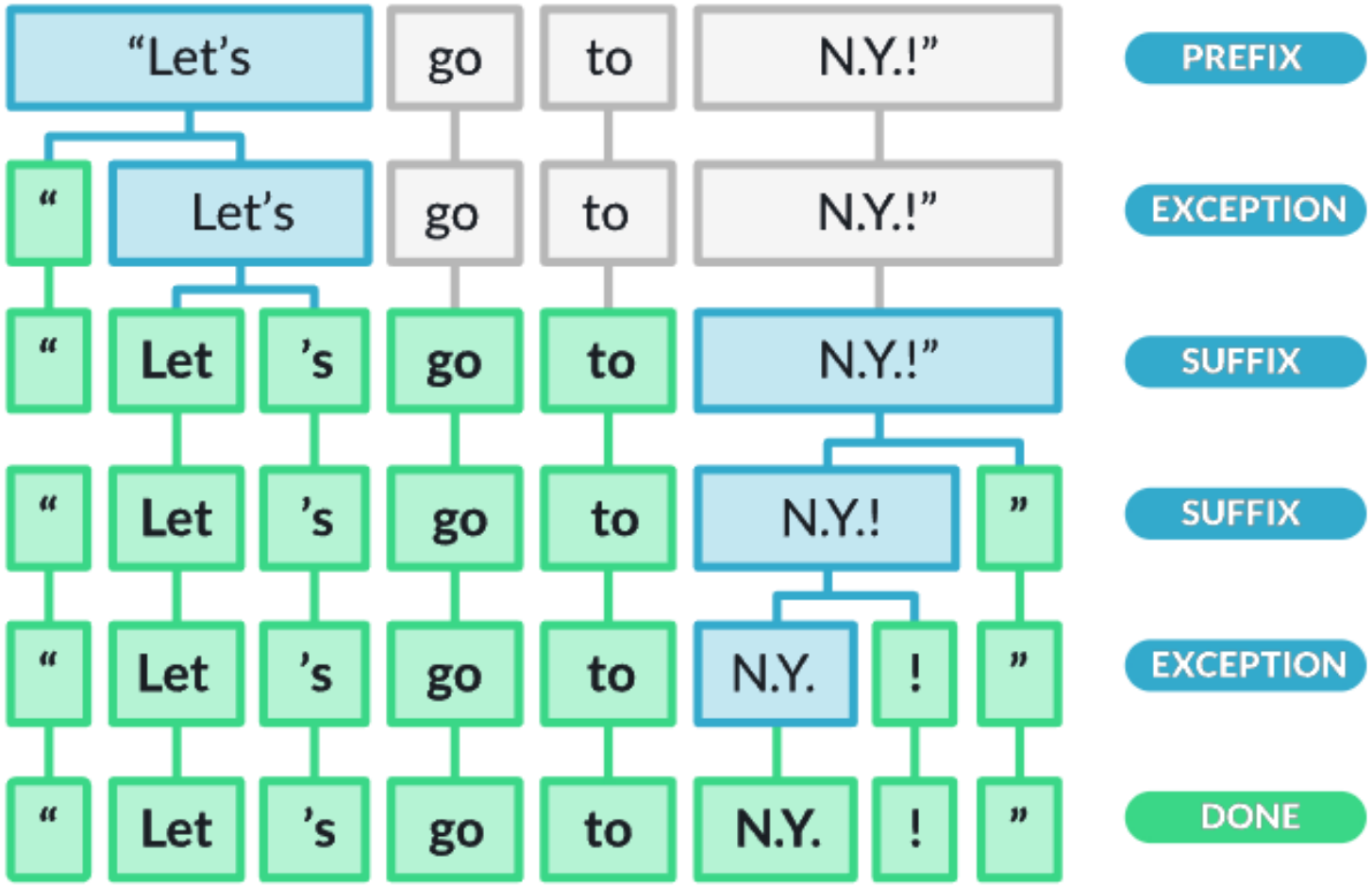
- doc = nlp('"Let\'s go to N.Y.!"')
  for token in doc:
  print(token.text)

- Bni mellal
- Bni ---- mellal

- If you need → dict → tokens
- Ngram
- Bigram
- If; yo ; ou; ne ; ee; ed

- nee ; eed ; ees
- nee d/s

Exemple


“Let’s
go
to
N.Y.!”
PREFIX
“
Let’s
go
to
N.Y.!”
EXCEPTION
“
Let
’s
go
to
N.Y.!”
SUFFIX
“
Let
’s
go
to
N.Y.!
”
SUFFIX
“
Let
’s
go
to
N.Y.
!
”
EXCEPTION
“
Let
’s
go
to
N.Y.
!
”
DONE

Named Entities with spaCy

- sentence = nlp(u'Google's monthly revenue is $6 million')

- for token in sentence:
- print(token.text, end=' | ')

- print('
- _____
_____')

- for ent in sentence.ents:
- print(ent.text+' - '+ent.label_+' -
'+str(spacy.explain(ent.label_)))

Visualizing named entity

- from spacy import displacy
  doc = nlp(u'Over last few years USA generates $6 million revenue.')
  displacy.render(doc, style='ent', jupyter=True)

Visualizing the dependency

- from spacy import displacy
- document = nlp(u'Bangladesh is a beautiful country')
- displacy.render(document, style='dep', jupyter=True, options={'distance': 110})

Stemming with NLTK

- import nltk
  from nltk.stem.porter import *
  p_stemmer = PorterStemmer()
  words =
  ['run','runner','running','ran','runs','easily','fairly']
  for word in words:
  print(word+' --> '+p_stemmer.stem(word))

Snowball Stemmer

- from nltk.stem.snowball import SnowballStemmer
- s_stemmer = SnowballStemmer(language='english')
- words = ['run','runner','running','ran','runs','easily','fairly']
- for word in words:
- print(word+' --> '+s_stemmer.stem(word))

Lemmatization and POS tagging with spaCy

- doc = nlp(u"He is a runner running in a competition because he loves to run since he ran today")

- for token in doc:
- print(f'{token.text:{12}} {token.pos_:{6}} {token.lemma:<{22}} {token.lemma_}')

Stop Words with spaCy

- import spacy
- nlp = spacy.load('en_core_web_sm')
- print(nlp.Defaults.stop_words)

Checking a specific token whether it is a stop word or not.

- nlp.vocab['myself'].is_stop

- nlp.vocab['mystery'].is_stop

A suivre